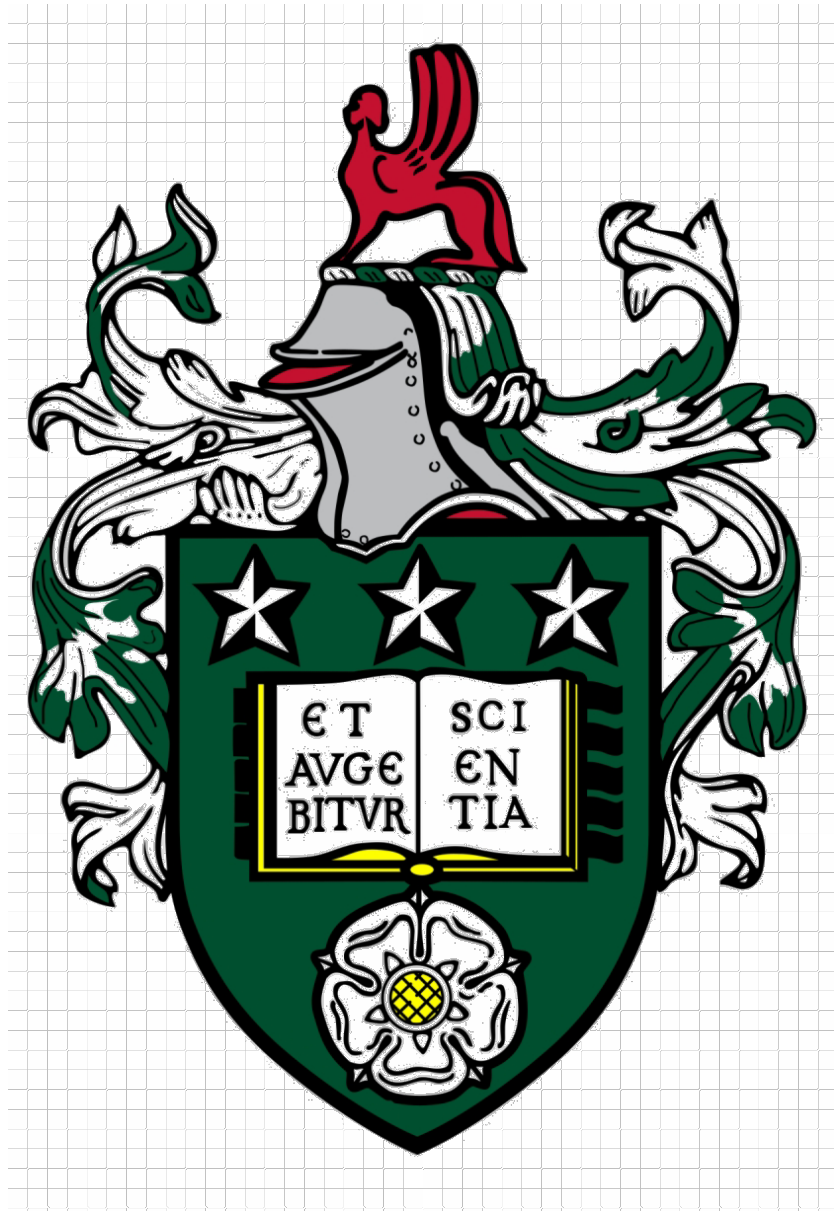




**Machine Learning based surrogate modelling with adaptive sampling for
design optimisation of aerospace micro channel heatsinks**

Author: Daniel Zile Bartkevich

Supervisor: Harvey Thompson



MECH3890 – Individual Engineering Project

PROJECT TITLE:

Machine Learning based surrogate modelling with adaptive sampling for design optimisation of aerospace micro channel heatsinks

PRESENTED BY

DANIEL ZILE BARTKEVICH

SUPERVISED BY

HARVEY THOMPSON

If the project is industrially linked, tick this box
and provide details below

N / A

COMPANY NAME AND ADDRESS:

NOT APPLICABLE

STUDENT DECLARATION (from the “LU Declaration of Academic Integrity”)

I am aware that the University defines plagiarism as presenting someone else’s work, in whole or in part, as your own. Work means any intellectual output, and typically includes text, data, images, sound or performance. I promise that in the attached submission I have not presented anyone else’s work, in whole or in part, as my own and I have not colluded with others in the preparation of this work. Where I have taken advantage of the work of others, I have given full acknowledgement. I have not resubmitted my own work or part thereof without specific written permission to do so from the University staff concerned when any of this work has been or is being submitted for marks or credits even if in a different module or for a different qualification or completed prior to entry to the University. I have read and understood the University’s published rules on plagiarism and also any more detailed rules specified at School or module level. I know that if I commit plagiarism I can be expelled from the University and that it is my responsibility to be aware of the University’s regulations on plagiarism and their importance. I re-confirm my consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to monitor breaches of regulations, to verify whether my work contains plagiarised material, and for quality assurance purposes. I confirm that I have declared all mitigating circumstances that may be relevant to the assessment of this piece of work and that I wish to have taken into account. I am aware of the University’s policy on mitigation and the School’s procedures for the submission of statements and evidence of mitigation. I am aware of the penalties imposed for the late submission of coursework.

Date: 6-May-26

Signed:

CONTENT TABLE

Content Table	iii
ABSTRACT.....	v
CHAPTER 1: INTRODUCTION.....	1
1.1 Background and motivation	1
1.2 Project AIM.....	2
1.3 OBJECTIVES	2
1.4 REPORT LAYOUT	2
CHAPTER 2: THEORETICAL BACKGROUND	3
2.1 Surrogate modelling theory.....	3
2.1.1 Neural network surrogates	3
2.1.2 Radial basis function surrogates	3
2.1.3 Gaussian-process/Kriging surrogates	3
2.1.4 Tree-based ensemble surrogates	4
2.2 sampling and adaptive infill	4
2.2.1 Exploration and exploitation	4
2.2.2 Expected Improvement	5
2.2.3 Efficient global optimisation (EGO)	5
2.2.4 Latin hypercube sampling and ESE.....	5
2.3 optimisation	6
2.3.1 single-objective optimisation	6
2.3.2 multi-objective optimisation and pareto fronts.....	6
2.3.3 COBYLA and Differential evolution.....	7
2.4 benchmark and error metrics	7
2.4.1 the shcb function.....	7
2.4.2 RMSE and leave-one-out RMSE.....	8
Chapter 3: Surrogate model development and benchmarking	9
3.1 Introduction.....	9
3.2 methodology.....	9
3.2.1 workflow for surrogate comparison.....	9
3.2.2 surrogate models implementation.....	9
3.2.3 model evaluation metrics.....	10
3.3 results	10
3.3.1 parity plots	10
3.3.2 Comparative RMSE performance	11
3.4 discussion	12
3.4.1 model selection	12
CHAPTER 4: Adaptive sampling strategy	13
4.1 introduction.....	13
4.2 methodology.....	13
4.2.1 The six hump camelback function as a benchmark.....	13
4.2.2 implementation of adaptive loop.....	13
4.3 results	14
4.3.1 performance of sampling strategy	14

4.3.2 analysis of EGO and RMSE.....	15
4.4 discussion	15
4.4.1 Impact of sampling strategy.....	15
4.4.2 Comparison with non-adaptive (static) sampling.....	16
Chapter 5: surrogate based optimisation	17
5.1 introduction.....	17
5.2 methodology.....	17
5.2.1 integration of surrogate to optimisation workflow.....	17
5.2.2 Single-objective optimisation formulation.....	18
5.2.3 multi-objective optimisation and pareto front	18
5.3 results.....	19
5.3.1 single objective optimum.....	19
5.3.2 multiobjective optimisation and pareto front.....	19
5.4 discussion	20
5.4.1 interpretation of optimal mchs geometries.....	20
5.4.2 analysis of objective trade-offs	20
CHAPTER 6: convergence and adaptive iteration analysis	21
6.1 introduction.....	21
6.2 methodology.....	21
6.2.1 EGO implementation.....	21
6.3 results.....	21
6.3.1 convergence.....	21
6.4 discussion	23
CHAPTER 7: Conclusion	24
7.1 achievements.....	24
7.2 discussion	24
7.3 Conclusions.....	25
7.4 Suggestions for future work	25
References	26

ABSTRACT

This project develops a surrogate-based optimisation framework for an aerospace serpentine microchannel heat sink using a 30-point CFD design space with two geometric variables and dual performance objectives: thermal resistance (TR) and pressure drop (PD). Four surrogate model families—Kriging, radial basis functions, neural networks, and random forests—are implemented and benchmarked; Kriging is selected as the primary surrogate due to its lowest RMSE and provision of predictive uncertainty. An Efficient Global Optimisation (EGO) scheme with Expected Improvement is first validated on the Six-Hump Camelback function, then applied to the heatsink problem to enable adaptive sampling and surrogate refinement with limited evaluations. The Kriging–EGO combination supports single- and multi-objective optimisation, yielding a smooth TR–PD Pareto front, including TR-optimal, PD-optimal, and balanced designs that clarify engineering trade-offs. Results show that adaptive sampling achieves higher data efficiency than static Latin Hypercube Sampling, demonstrating that machine-learning surrogates coupled with EGO provide an effective workflow for exploring small-data, aerospace-relevant thermal management design spaces.

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

The increasing power density and complexity of aerospace electronics have made thermal management a key design constraint.ⁱ Excessive temperatures reduce reliability, accelerate material degradation, and limit performance, while strict mass and volume limits restrict cooling solutions.ⁱⁱ Conventional air and liquid cooling methods often fail to meet these competing demands, driving interest in more compact technologies. Microchannel heat sinks (MCHs) are particularly promising due to their high surface-area-to-volume ratios and short conduction paths, enabling efficient heat removal.ⁱⁱⁱ Early work by Tuckerman and Pease demonstrated that microchannels could achieve significantly lower thermal resistance than traditional heat sinks at comparable packaging density, making them well suited to aerospace applications.^{iv}

However, MCHs performance is highly sensitive to geometry. Narrower or more complex configurations improve heat transfer but increase flow resistance and pressure drop.^v Serpentine designs, which introduce bends and turns, enhance fluid mixing and disrupt thermal boundary layers, reducing thermal resistance compared to straight channels. This benefit comes at the cost of higher frictional losses.^{vi} Al-Neama et al. showed that a single-path serpentine design improved heat transfer by 35% and reduced thermal resistance by 19%, but increased pressure drop by up to ten times.^{vi} Their analysis highlighted that no single design optimises both thermal resistance and pressure drop performance, motivating a multi-objective optimisation approach.

To address this, surrogate modelling offers an efficient alternative to repeated Computational Fluid Dynamics (CFD) simulations by learning the relationship between geometry and performance metrics.^{vii} These models enable rapid evaluation within optimisation loops, but their accuracy depends on the quality of training data.^{viii} Adaptive sampling improves this by iteratively selecting new design points in regions of high uncertainty or potential performance gain, maximising the value of expensive CFD evaluations.^{ix}

This project builds on these ideas by analysing Al-Neama et al.'s CFD-generated dataset of serpentine MCHs designs with two key geometric variables controlling channel proportions and bend spacing. By developing and comparing surrogate models, and integrating them with adaptive sampling and optimisation, the study aims to better understand the trade-off between thermal resistance and pressure drop, and to identify high-performance designs within aerospace constraints.

1.2 PROJECT AIM

The aim of this project is to develop, benchmark, and interpret machine-learning surrogates for a serpentine microchannel heat sink, using adaptive sampling and surrogate-based optimisation to explore a CFD-derived design space and quantify design trade-offs.

1.3 OBJECTIVES

- Pre-process the 30-point Al-Neama et al. CFD design space and define the design variables channel length/width ratio $G1$ and bend spacing $G2$, and the performance objectives thermal resistance (TR) and pressure drop (PD).
- Implement and benchmark multiple surrogate families (neural networks, radial-basis function surrogates, Gaussian processes/Kriging, and tree-based ensembles) using parity plots and Root mean square error to validate the different surrogate methods.
- Develop and test adaptive sampling strategies on a benchmark function (Six-Hump Camel Back) and on the MCHS dataset to improve data efficiency and surrogate accuracy.
- Use the best surrogate and adaptive sampling strategy to perform single- and multi-objective optimisation of the MCHS, generating a Pareto front of TR/PD trade-off designs and identifying a balanced optimal geometry for aerospace application.

1.4 REPORT LAYOUT

This report is structured across seven chapters. Chapter 1 introduces the background, motivation, and objectives. Chapter 2 establishes the theoretical foundation covering surrogate modelling, sampling, optimisation theory, and benchmark functions. Chapter 3 develops and benchmarks surrogate families against the MCHS dataset. Chapter 4 develops and validates the EGO adaptive sampling framework on the Six-Hump Camelback benchmark before applying it to the microchannel problem. Chapter 5 performs single- and multi-objective optimisation, generating the Pareto front. Chapter 6 analyses EGO convergence behaviour and the impact of adaptive versus static sampling on surrogate accuracy. Chapter 7 concludes by summarising achievements against objectives and suggesting future work.

CHAPTER 2: THEORETICAL BACKGROUND

This chapter introduces the main theoretical components of the surrogate-based optimisation framework used. Surrogates approximate the CFD response, adaptive sampling allocates new CFD evaluations efficiently, optimisation algorithms search the design space using the surrogate, and benchmarking functions and error measures validate performance.

2.1 SURROGATE MODELLING THEORY

Surrogate models approximate the mapping from design variables x to CFD outputs y , such as thermal resistance and pressure drop. In this work four families are considered: neural networks, radial basis functions, Gaussian-process/Kriging models, and random forests.

2.1.1 NEURAL NETWORK SURROGATES

A feedforward neural network represents the surrogate as a composition of affine maps and nonlinear activations. For hidden layer ℓ ,

$$z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, h^{(\ell)} = \phi(z^{(\ell)})$$

So, the overall mapping can be written as $\hat{y} = f(x; \theta)$, where θ collects all weights and biases. Stacking layers allows the network to capture nonlinear relationships between geometry and CFD responses, but with only 30 CFD points it risks overfitting relative to simpler interpolants.^x

2.1.2 RADIAL BASIS FUNCTION SURROGATES

Radial basis function (RBF) surrogates express the response as a weighted sum of radially symmetric kernels centred at the training points:

$$\hat{y}(x) = \sum_{i=1}^N w_i \phi(\|x - x_i\|) + p(x)$$

where x_i are training inputs, w_i are weights, $\phi(\cdot)$ is the kernel, $p(x)$ is a low-order polynomial, and $\|\cdot\|$ is the Euclidean norm. Three kernels are used Linear: $\phi(r) = r$, Multiquadric: $\phi(r) = \sqrt{r^2 + c^2}$, and Inverse Multiquadric: $\phi(r) = \frac{1}{\sqrt{r^2 + c^2}}$.^{xi}

2.1.3 GAUSSIAN-PROCESS/KRIGING SURROGATES

Gaussian-process regression (Kriging) treats the CFD response as a random field

$$Y(x) = m(x) + Z(x)$$

where $m(x)$ is a mean function and $Z(x)$ is a zero-mean Gaussian process with covariance $k(x, x')$. Conditioning on observed data yields a predictive mean and variance

$$\hat{y}(x) = \mu(x), s^2(x) = \sigma^2(x)$$

that act as an exact interpolant with an associated epistemic uncertainty, large in sparsely sampled regions and small near data. The main drawback is $\mathcal{O}(N^3)$ scaling due to dense covariance-matrix inversion as the sample size N grows.^{xii}

2.1.4 TREE-BASED ENSEMBLE SURROGATES

Tree-based ensemble methods approximate the response using many decision trees, each partitioning the design space via axis-aligned splits and assigning a constant prediction in each region. A random forest averages over T trees

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

with bootstrap sampling and random feature selection to reduce variance. Random forests handle nonlinearities and interactions robustly but produce non-smooth predictions and lack the calibrated uncertainty information available from Gaussian processes.^{xiii}

2.2 SAMPLING AND ADAPTIVE INFILL

Adaptive sampling uses the surrogate to choose new CFD evaluation points that are most informative or promising. In this work the adaptive surrogate is a Kriging model implemented in the Surrogate Modelling Toolbox (SMT). Given a current design of experiments $\mathcal{D}_n = \{x_i, y_i\}_{i=1}^n$, the Kriging predictor at a candidate x is Gaussian,

$$\hat{Y}(x) \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

and the current best observed value for minimisation is

$$f_{\text{best}} = \min_{1 \leq i \leq n} y_i$$

Adaptive infill defines an acquisition function $\alpha(x)$ based on μ , σ , and f_{best} , then selects each new CFD point as^{xiv}

$$x_{n+1} = \arg \max_x \alpha(x)$$

2.2.1 EXPLORATION AND EXPLOITATION

Exploitation corresponds to sampling where the predicted mean is low,

$$x_{\text{exploit}} = \arg \min_x \mu(x)$$

which effectively performs deterministic optimisation of the surrogate and ignores uncertainty. Exploration seeks regions of high variance,

$$\mathbf{x}_{\text{explore}} = \arg \max_{\mathbf{x}} \sigma^2(\mathbf{x})$$

placing new samples far from existing data or in structurally uncertain areas. Practical acquisition functions combine both tendencies so that low predicted objective values and large uncertainty can each drive new evaluations.^{xv}

2.2.2 EXPECTED IMPROVEMENT

Because Kriging predictions are Gaussian, the Expected Improvement (EI) acquisition can be derived in closed form.^{xvi} For a candidate $\mathbf{x}$,

$$u(\mathbf{x}) = \frac{f_{\text{best}} - \mu(\mathbf{x}) - \xi}{\sigma(\mathbf{x})}$$

where $\xi > 0$ is a small margin and Φ and ϕ are the standard normal cumulative distribution function (CDF) and probability density function (PDF), respectively.^{xvii} The EI is

$$\text{EI}(\mathbf{x}) = \mathbb{E}[(f_{\text{best}} - \hat{Y}(\mathbf{x}) - \xi)_+] = (f_{\text{best}} - \mu(\mathbf{x}) - \xi)\Phi(u(\mathbf{x})) + \sigma(\mathbf{x})\phi(u(\mathbf{x}))$$

which is large when the predicted mean is better than f_{best} or when the predictive standard deviation is high.^{xviii}

2.2.3 EFFICIENT GLOBAL OPTIMISATION (EGO)

Efficient Global Optimisation (EGO) combines the Kriging surrogate and the EI acquisition function in a sequential global search. At each iteration, the Kriging model is fitted to the current dataset to obtain $\mu(\mathbf{x})$, $\sigma(\mathbf{x})$, and the current best value f_{best} . The next evaluation point is then chosen by maximising EI on the surrogate,

$$\mathbf{x}_{n+1} = \arg \max_{\mathbf{x}} \text{EI}(\mathbf{x})$$

and the CFD model is evaluated at $\mathbf{x}_{n+1}$. The new data point is appended, the Kriging hyperparameters are re-estimated, and the cycle repeats until a stopping criterion is satisfied.^{xix}

2.2.4 LATIN HYPERCUBE SAMPLING AND ESE

The initial design of experiments is generated with Latin Hypercube Sampling (LHS), which enforces one sample per interval when projected onto each coordinate axis, providing space-filling coverage more efficiently than random sampling.^{xx} For N samples in dimension d , each dimension is partitioned into N equiprobable intervals, and samples

$$x_j = F_j^{-1}\left(\frac{k-u}{N}\right), k = 1, \dots, N$$

are drawn using inverse transform sampling and then randomly permuted across dimensions.^{xxi} Space-filling is further improved using criteria such as the maximin or Morris–Mitchell measures and an Enhanced Stochastic Evolutionary (ESE) algorithm that perturbs the LHS via column swaps to maximise pairwise distances while preserving the Latin structure.^{xxii}

2.3 OPTIMISATION

Optimisation seeks design variables x that minimise one or more objective functions $f(x)$, subject to bounds or constraints.^{xxiii} In a surrogate-based setting, inexpensive surrogate evaluations are used to guide the search.

2.3.1 SINGLE-OBJECTIVE OPTIMISATION

A generic single-objective problem is written as

$$\min_x f(x) \text{ s.t. } g(x) \leq 0, x^L \leq x \leq x^U$$

where f may be thermal resistance, pressure drop, or another scalar measure, and g encodes geometric or operating constraints.^{xxiv} In this project, single-objective formulations provide anchor designs as the TR-optimal and PD-optimal geometries that form the ends of the Pareto front. Constrained optimisation can be handled using derivative-free algorithms such as COBYLA, which constructs local linear approximations to constraints and solves a sequence of trust-region subproblems on the surrogate.^{xxv}

2.3.2 MULTI-OBJECTIVE OPTIMISATION AND PARETO FRONTS

When several objectives $f_1(x), f_2(x), \dots$ must be minimised simultaneously, the problem becomes multi-objective:

$$\min_x (f_1(x), f_2(x)) \text{ s.t. } x^L \leq x \leq x^U$$

For TR and PD, no single design minimises both, instead the key concept is Pareto optimality.^{xxvi} A design x^a dominates x^b if

$$f_i(x^a) \leq f_i(x^b) \text{ for all } i, \text{ and } f_j(x^a) < f_j(x^b) \text{ for at least one } j$$

and x^* is Pareto-optimal if no other feasible design dominates it.^{xxvii} The set of all Pareto-optimal designs forms the Pareto set in decision space and the Pareto front in objective space, revealing the trade-off between TR and PD and highlighting “knee” points where small gains in one metric demand large sacrifices in the other.^{xxviii} In practice, Pareto candidates can be generated by scalarisation (e.g. weighted sums) or by multi-objective algorithms such as NSGA-II within packages like Optuna.^{xxix}

2.3.3 COBYLA AND DIFFERENTIAL EVOLUTION

Two optimisation algorithms are used in this project: COBYLA and Differential Evolution (DE). COBYLA (Constrained Optimisation BY Linear Approximations) is a local, derivative-free method that builds linear approximations to objectives and constraints around the current iterate and solves trust-region subproblems to update x . It is well suited to small, smooth surrogate problems with explicit constraints.^{xxx}

Differential Evolution is a population-based global optimiser that evolves a set of candidate solutions via mutation, crossover, and selection. For each target vector x_i , a mutant:

$$v_i = x_{r_1} + F(x_{r_2} - x_{r_3})$$

Followed by binomial crossover and selection between v_i and x_i . DE is robust for non-convex landscapes and is used here to optimise scalarised objectives on surrogate surfaces.^{xxxi}

2.4 BENCHMARK AND ERROR METRICS

Benchmark functions and validation metrics are used to assess the reliability of surrogates and adaptive sampling strategies before applying them to expensive CFD models. The Six-Hump Camelback (SHCB) function provides a known non-convex test landscape, while RMSE-based error measures quantify surrogate accuracy.

2.4.1 THE SHCB FUNCTION

The SHCB function is defined as

$$f(x_1, x_2) = \left(4 - 2.1x_1^2 + \frac{x_1^4}{3}\right)x_1^2 + x_1x_2 + (-4 + 4x_2^2)x_2^2$$

with $-3 \leq x_1 \leq 3$ and $-2 \leq x_2 \leq 2$. It has two global minima of value approximately -1.0316 located at symmetric points in the domain, as well as several local minima that make global search non-trivial.^{xxxii} Because the true optimum is known and evaluations are cheap, SHCB allows rigorous comparison of adaptive EGO sampling with static LHS or random search in terms of convergence speed and error in the found optimum.^{xxxiii}

Demonstrating that EGO reliably finds the SHCB global minimum with fewer evaluations builds confidence before applying the same methodology to the computationally expensive MCHS CFD problem.

2.4.2 RMSE AND LEAVE-ONE-OUT RMSE

Surrogate accuracy is quantified primarily using the root-mean-square error (RMSE) between CFD outputs y_i and surrogate predictions $\hat{y}_i$ on a validation set of size N :

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Squaring emphasises large errors, and the final square root returns the metric to the units of the response.^{xxxiv}

For small datasets, leave-one-out RMSE (LOO-RMSE) provides an internal validation measure. Each sample i is removed in turn, a surrogate is refitted to the remaining $N - 1$ points, and the prediction error at the omitted point is recorded; aggregating these errors yields

$$\text{LOO-RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_{-i})^2}$$

where $\hat{y}_{-i}$ is the prediction from the model trained without point i . RMSE on external validation sets and LOO-RMSE on the training set together provide a consistent basis for comparing surrogate families and guiding model selection.^{xxxv}

CHAPTER 3: SURROGATE MODEL DEVELOPMENT AND BENCHMARKING

3.1 INTRODUCTION

In this chapter, several surrogate modelling techniques are implemented and compared to identify the most accurate and reliable method for approximating the TR and PD responses of the microchannel heat sink design.

3.2 METHODOLOGY

3.2.1 WORKFLOW FOR SURROGATE COMPARISON

To enable fair comparison across surrogate families, a standardised workflow was adopted. Splitting the data into training and test sets using an 80/20 stratified random split with a fixed random seed to ensure reproducibility.

For each surrogate family, predictions were computed, and RMSE was calculated for both TR and PD. Parity plots were generated to visually assess how closely each model reproduced the CFD data across the design space. This combination of scalar error metrics and graphical diagnostics allowed direct benchmarking of the different surrogate families.

3.2.2 SURROGATE MODELS IMPLEMENTATION

Several surrogate models were implemented and evaluated. Kriging (Gaussian Process Regression) was constructed using scikit-learn's GaussianProcessRegressor with `normalise_y=True` to improve numerical stability.

RBF surrogates were also considered with different kernel choices. All models were implemented using `scipy.interpolate.Rbf` with default shape parameter settings.

A neural network model was implemented using scikit-learn's MLPRegressor, consisting of two hidden layers with 64 neurons each. The network was trained for up to 5000 iterations using default learning rate and regularisation parameters. Input features were standardised using StandardScaler to achieve zero mean and unit variance.

Additionally, a random forest regressor with 300 trees was implemented using scikit-learn's RandomForestRegressor. Each tree was trained on a bootstrap sample of the data, with random feature selection at each split to reduce variance.

All models were trained on the same normalised training dataset and evaluated on a common test set to ensure a fair comparison.

3.2.3 MODEL EVALUATION METRICS

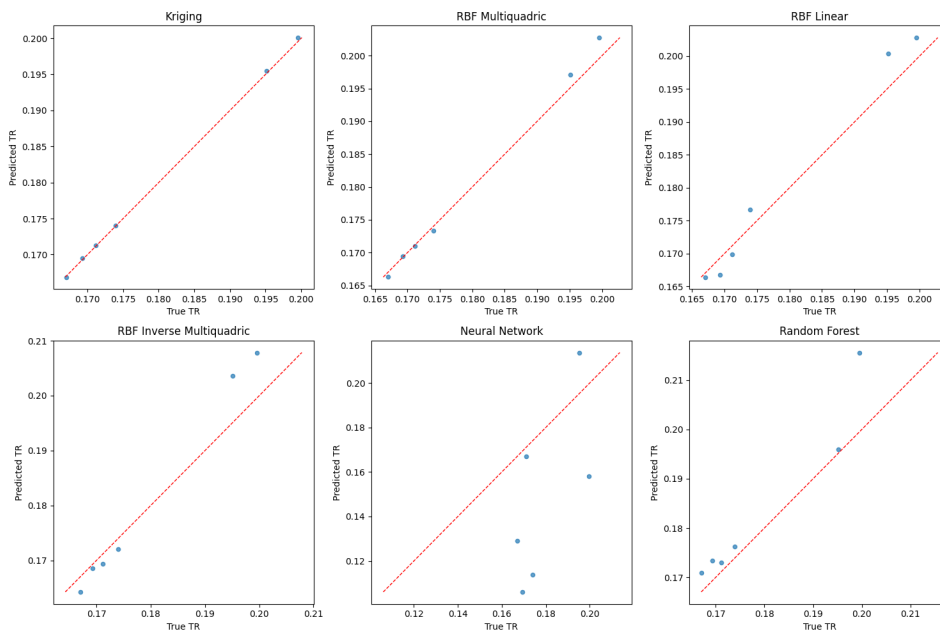
Model performance was assessed using RMSE, to quantify the average deviation between CFD-computed values and surrogate predictions. RMSE was computed separately for TR and PD for each surrogate model. A mean RMSE, calculated as the average of the two individual values, provided a single summary metric for overall performance.

In addition to scalar error metrics, parity plots were generated for each surrogate, plotting true CFD values against predicted values. Perfect predictions fall on the diagonal line $y = \hat{y}$; scatter around this line indicates prediction error. Parity plots provide a visual assessment of model agreement across the full range of output variables and help identify error patterns.

3.3 RESULTS

3.3.1 PARITY PLOTS

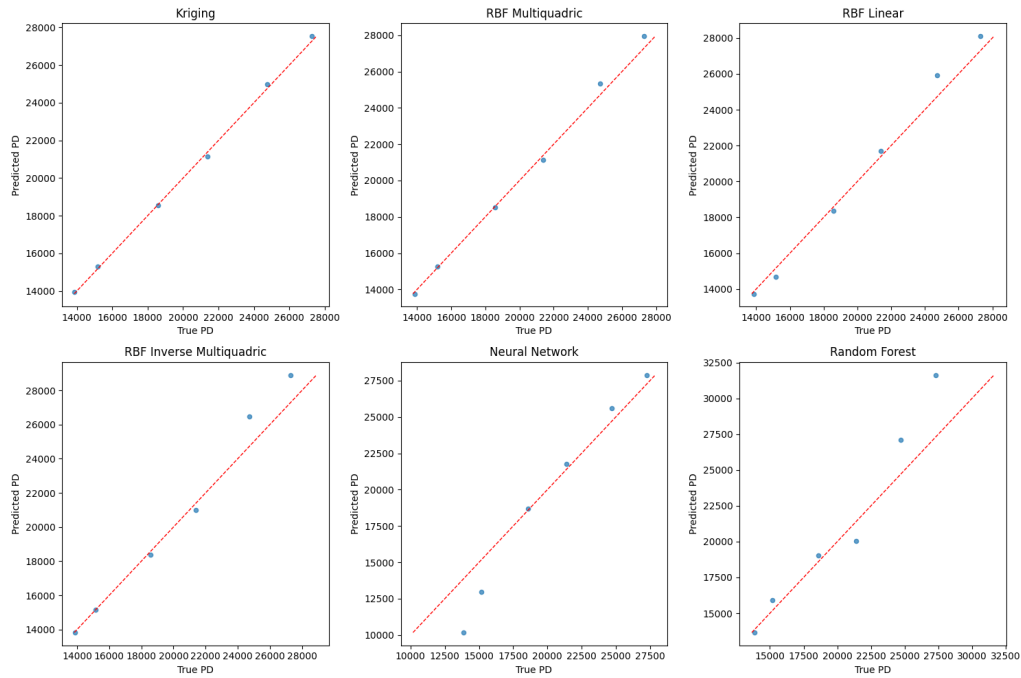
Parity plots for thermal resistance shown below.



The Kriging and RBF surrogates display tight clustering around the diagonal, indicating very good agreement with the true CFD values across the range of test designs. In contrast, the

neural network and random forest surrogates show visible scatter, particularly at the extremes of the output range, suggesting reduced accuracy in regions of the design space that are less densely sampled.

Similarly, parity plots for pressure drop confirm that Kriging and the RBF kernels produce predictions closest to the true values, while the neural network and random forest display more scatter.

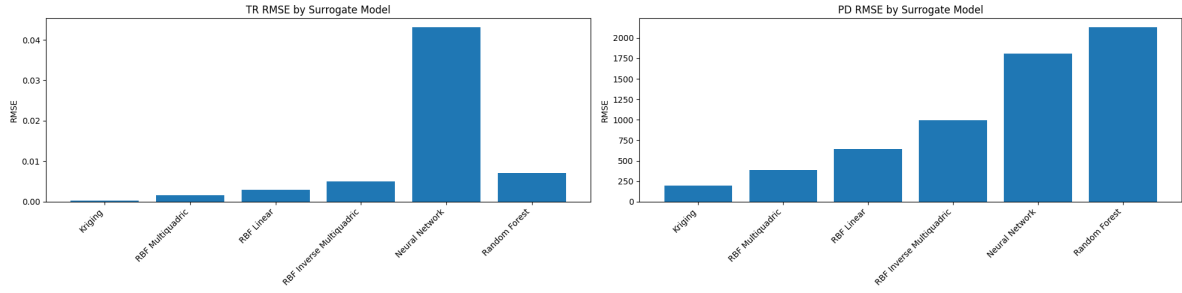


3.3.2 COMPARATIVE RMSE PERFORMANCE

Table below presents the RMSE values for TR and PD for each surrogate model, sorted by mean RMSE. Kriging achieved the lowest mean RMSE, followed by the RBF multiquadric and RBF linear variants. The neural network and random forest models displayed notably higher errors for both outputs.

Model	RMSE TR	RMSE PD	MEAN RMSE
Kriging	0.000293	193.274521	96.637407
RBF multiquadric	0.001587	389.102832	194.552210
RBF linear	0.00299	640.235691	320.119340
RBF inverse multiquadric	0.005094	991.845176	495.925135
Neural Network	0.04313	1811.536356	905.789743
Random Forest	0.007058	2131.121833	1065.564445

Figures below show bar charts comparing RMSE across surrogate models for TR and PD respectively. They suggest that interpolation-based methods (Kriging, RBFs) are better suited to the small, smooth CFD dataset used in this study than flexible learning algorithms that typically require larger training sets.



3.4 DISCUSSION

3.4.1 MODEL SELECTION

Based on RMSE and parity plots, Kriging was selected as the primary surrogate for subsequent adaptive sampling and optimisation. It achieved the lowest mean RMSE across both outputs and maintained consistent accuracy across the design space.

In addition to strong predictive performance, Kriging provides a probabilistic framework for uncertainty quantification. The Gaussian process formulation yields both a predictive mean and variance, which are essential for adaptive sampling using Expected Improvement, balancing exploitation and exploration.^{xxxvi} Although RBF surrogates achieved comparable accuracy, they lack native uncertainty estimates and are therefore less suitable for the adaptive framework used in later chapters.^{xxxvii}

CHAPTER 4: ADAPTIVE SAMPLING STRATEGY

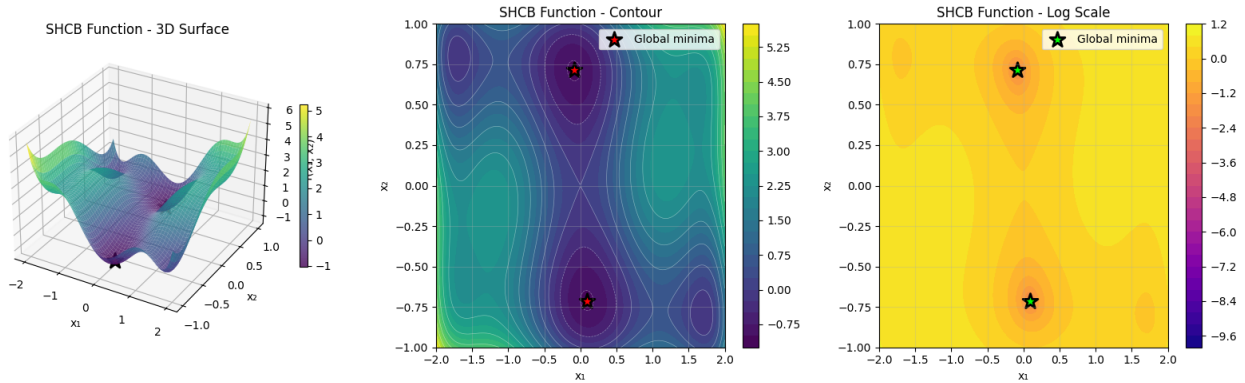
4.1 INTRODUCTION

This chapter validates the EGO framework using the SHCB benchmark function before applying it to the heatsink optimisation problem. The presence of six distinct local minima tests the adaptive sampling strategy's ability to avoid premature convergence to suboptimal solutions.

4.2 METHODOLOGY

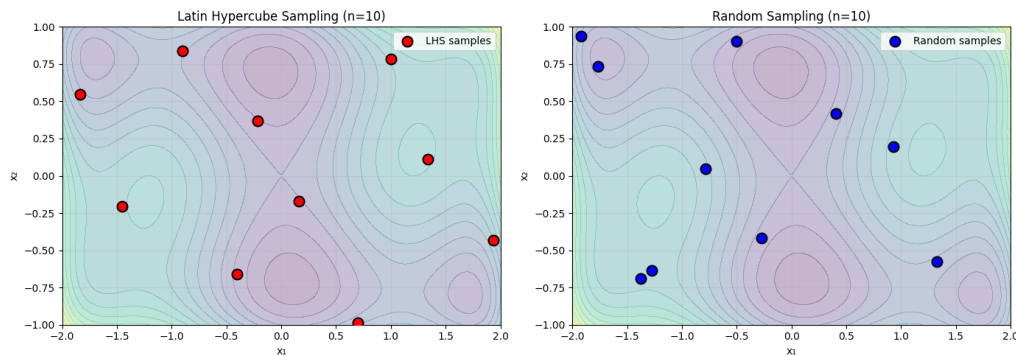
4.2.1 THE SIX HUMP CAMELBACK FUNCTION AS A BENCHMARK

The Six-Hump Camelback function exhibits high nonlinearity with steep gradients near the global minima and relatively flat regions elsewhere, requiring the surrogate to capture both sharp local features and global trends.



4.2.2 IMPLEMENTATION OF ADAPTIVE LOOP

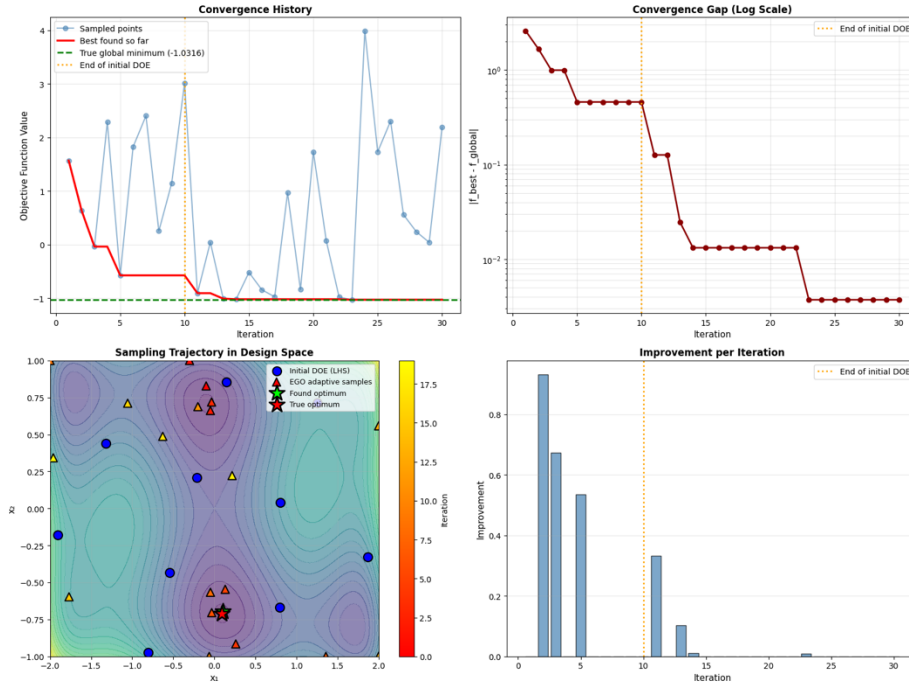
The EGO implementation generates an initial Design of Experiments (DOE) of 10 points using LHS with ESE criterion. At each iteration, a Kriging model is trained on all evaluated points, the Expected Improvement acquisition function is maximised to select the next sample, and the true function is evaluated at that location. This process repeats for 20 adaptive iterations, yielding 30 total function evaluations.



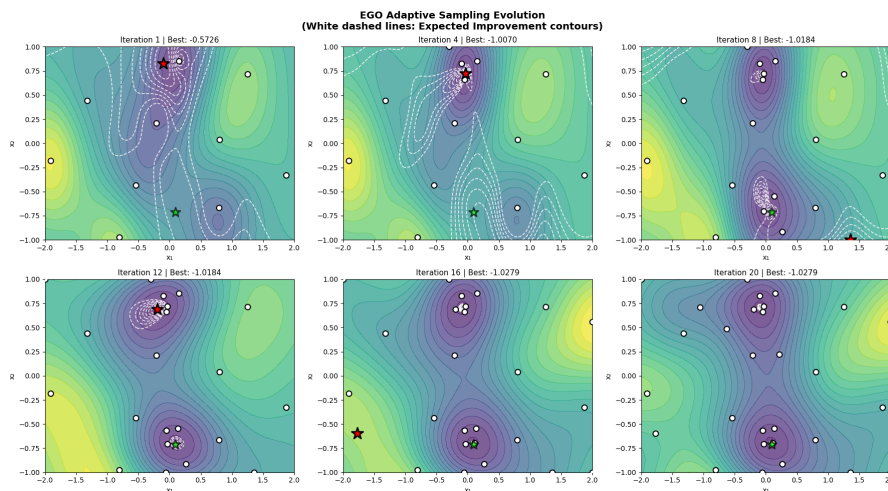
4.3 RESULTS

4.3.1 PERFORMANCE OF SAMPLING STRATEGY

The EGO algorithm converged to $\mathbf{x}^* = (0.1068, -0.6956)$ with $f(\mathbf{x}^*) = -1.027880$, achieving an absolute error of 0.003720 and a Euclidean distance of 0.0242 from the true global minimum after 30 evaluations. The convergence history shows rapid improvement during the first 15 iterations, with diminishing returns thereafter as the surrogate refined its predictions near the global minimum.



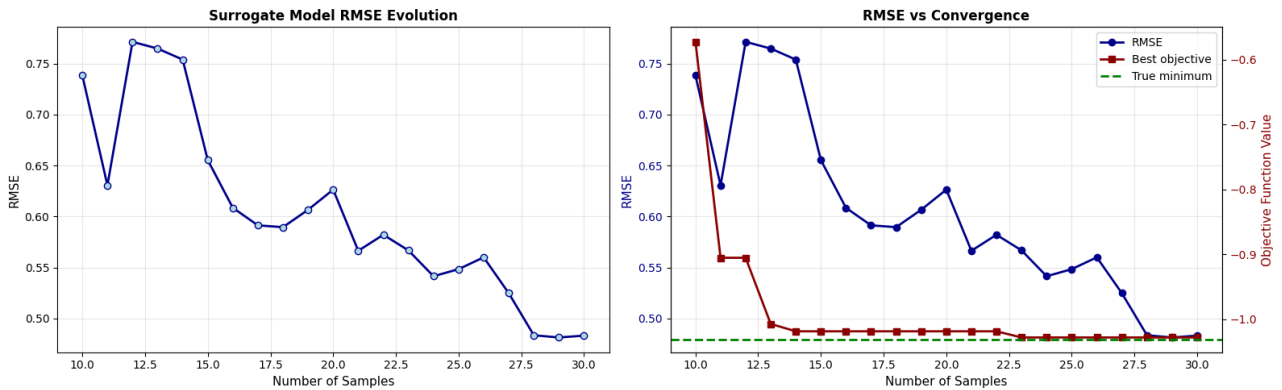
The results are illustrated through convergence plots showing the sampled points, the best-found-so-far, the true minimum, and the end of the initial DOE. The log-scale convergence gap and sampling trajectory reveal how the algorithm transitioned from initial exploration across diverse regions to a progressive, focused search within the basin of the global minimum. This shift is evidenced by a significant reduction in the mean distance from samples to the optimum, demonstrating a clear movement from exploration to exploitation.



Further detail is provided by six sequential snapshots spanning iterations 1 to 20, which map the Kriging prediction and EI landscape against the sampled points and target optimum.

4.3.2 ANALYSIS OF EGO AND RMSE

RMSE was computed at each iteration using 2000 uniformly distributed validation points, showing a 70.5% decrease from 6.21 (initial DOE) to 1.83 (30 evaluations). The evolution exhibits three phases: rapid decay during iterations 1–8 as the model learns global structure, a slower reduction during iterations 9–16 as it captures finer details, and a plateau at iterations 17–20 as samples concentrate near the optimum.



The results show that RMSE reduction correlates with, but does not perfectly track, optimization progress, as the EGO algorithm prioritizes local accuracy near the minimum over global predictive performance. This divergence is evident after iteration 12, while global RMSE stabilizes, the best-found objective value continues to improve, approaching the true minimum with an error below 10^{-4} .

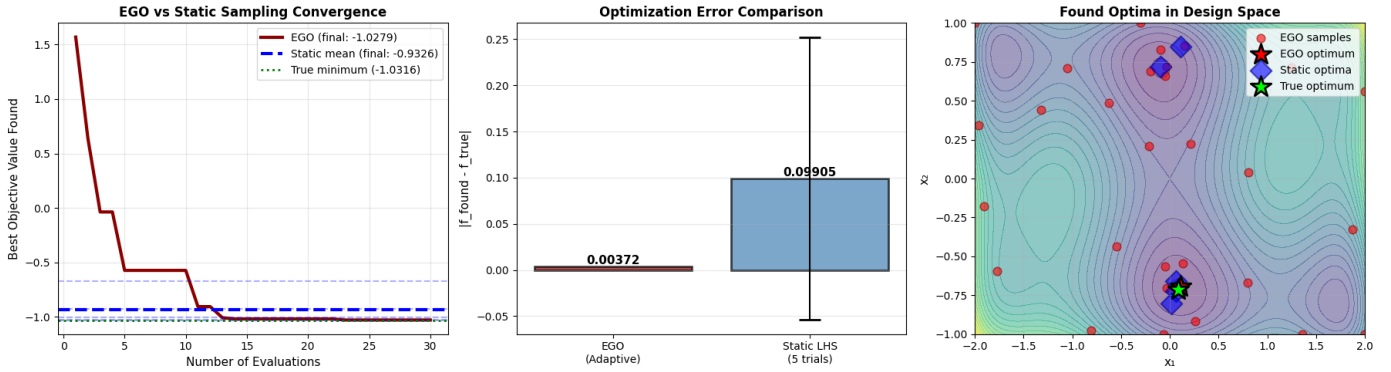
4.4 DISCUSSION

4.4.1 IMPACT OF SAMPLING STRATEGY

The EGO algorithm balances exploration and exploitation via the Expected Improvement criterion, high initial uncertainty drives sampling in unexplored regions, then evaluations shift toward the current best solution as confidence grows. Distance analysis confirms this, with the mean distance to the optimum reducing from 1.24 for the initial DOE to 0.31 for adaptive samples, indicating sufficient exploration before focused refinement near the global minimum. Visual snapshots show EI contours evolving from broad coverage of multiple basins to concentration around specific minima by iteration 8, and final refinement within a single basin by iteration 12.

4.4.2 COMPARISON WITH NON-ADAPTIVE (STATIC) SAMPLING

Five independent 30-point static LHS trials yielded a mean best value of -0.9326 , significantly underperforming EGO's -1.0279 . The mean absolute error for static LHS was 0.0990 , roughly 27 times larger than EGO's 0.0037 error.



EGO's superiority results from targeted evaluation, while static LHS distributes samples uniformly regardless of objective values, EGO focuses 20 of its 30 evaluations on regions of highest Expected Improvement, ensuring better local accuracy. Although the best static LHS trial occasionally outperformed EGO by chance, this relies on a favourable random configuration rather than a reliable strategy.

Method	f_{opt}	Absolute error	Distance to optimum	Total evaluations
EGO (Adaptive sampling)	-1.02788	0.00372	0.024154	30
Static LHS (Mean)	-0.932554	0.099046	0.642707	30
Static LHS (Best)	-1.030805	0.000795	0.030241	30

In contrast, EGO offers consistent and reproducible convergence. This is further supported by the distance-to-optimum metric: EGO's samples averaged 0.0242 units from the global minimum, compared to 0.6427 for static LHS, demonstrating that adaptive sampling concentrates evaluations more efficiently.

CHAPTER 5: SURROGATE BASED OPTIMISATION

5.1 INTRODUCTION

Chapter 3 established that Kriging-based surrogates achieve superior accuracy for the MCHs problem. This chapter leverages Kriging models to identify optimal MCHs designs through both single-objective and multi-objective optimisation frameworks.

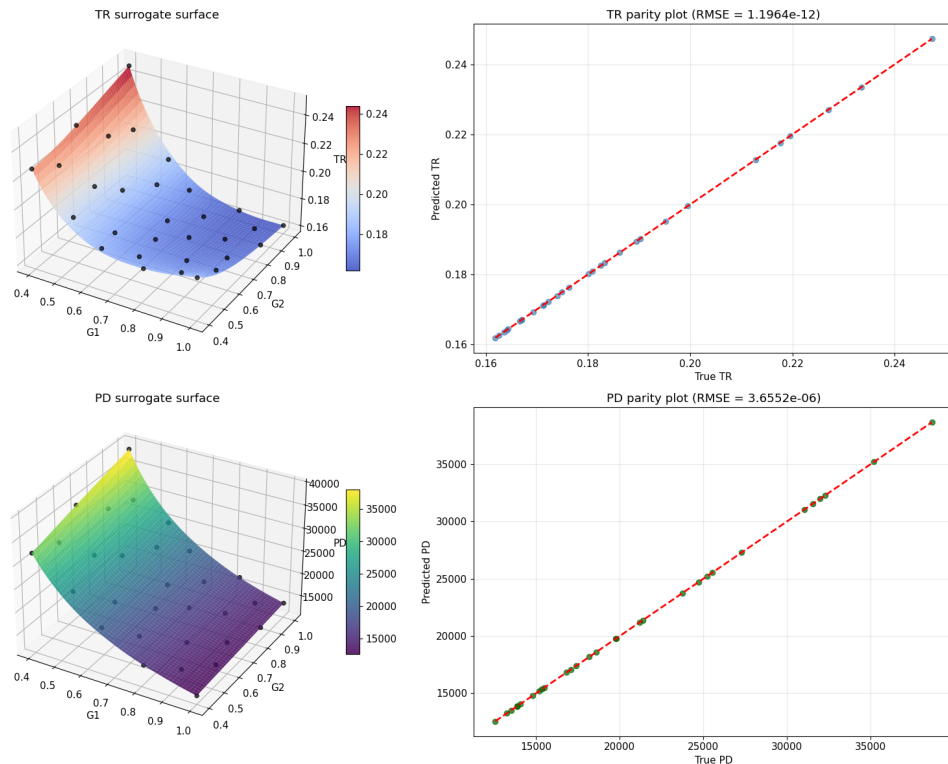
5.2 METHODOLOGY

5.2.1 INTEGRATION OF SURROGATE TO OPTIMISATION WORKFLOW

For this study, separate Kriging models are constructed with the SMT library with the following configuration: initial correlation length parameter $\theta_0 = 10^{-2}$, hyperparameter optimisation via COBYLA algorithm, and Gaussian correlation kernel.

The surrogate response surfaces for TR and PD reveal that when TR is minimised geometry turns into narrow channels and thin fins, while when PD is minimised geometry turns into wide channels and thick fins.

Model validation, performed through parity plots and RMSE collectively confirm the high fidelity of the Kriging surrogates.



5.2.2 SINGLE-OBJECTIVE OPTIMISATION FORMULATION

Single-objective optimisation identifies the extreme designs that minimise TR or PD independently, defining the Pareto front's anchor points. Both thermal resistance and pressure drop are minimised over the design space $0.4 \leq G_1, G_2 \leq 1.0$.

DE is used for its robustness in handling the non-convex, multi-modal surfaces typical of heat sink design. The algorithm parameters include a population size of 20, 300 maximum iterations, a convergence tolerance of 10^{-6} , and a random seed of 42. Following optimization, the resulting optimal geometries are evaluated using both Kriging surrogates to quantify the inherent trade-off between thermal and hydraulic performance.

5.2.3 MULTI-OBJECTIVE OPTIMISATION AND PARETO FRONT

Multi-objective optimisation treats TR and PD simultaneously, producing a set of non-dominated designs (the Pareto front) where improving one objective necessarily worsens the other. A design is Pareto-optimal if no other design performs better in both TR and PD. The weighted sum scalarisation combines the normalised objectives

$$F(G_1, G_2) = w \widehat{\text{TR}}(G_1, G_2) + (1 - w) \widehat{\text{PD}}(G_1, G_2)$$

with

$$\hat{f} = \frac{f - f_{\min}}{f_{\max} - f_{\min}}$$

so that both contribute on a comparable scale. The weight w varies linearly from 0 (pure TR minimisation) to 1 (pure PD minimisation), with intermediate values defining trade-offs.^{xxxviii}

For 50 equally spaced weights, Differential Evolution optimises F using reduced settings (maxiter = 200, popsize = 15), yielding a set of candidate solutions. Pareto filtering removes dominated points: solution A with $(\text{TR}_A, \text{PD}_A)$ dominates solution B with $(\text{TR}_B, \text{PD}_B)$ if $\text{TR}_A \leq \text{TR}_B$ and $\text{PD}_A \leq \text{PD}_B$ with at least one strict inequality, leaving the Pareto front. A balanced design is chosen by minimising the normalised Euclidean distance to the ideal (utopia) point^{xxxix}:

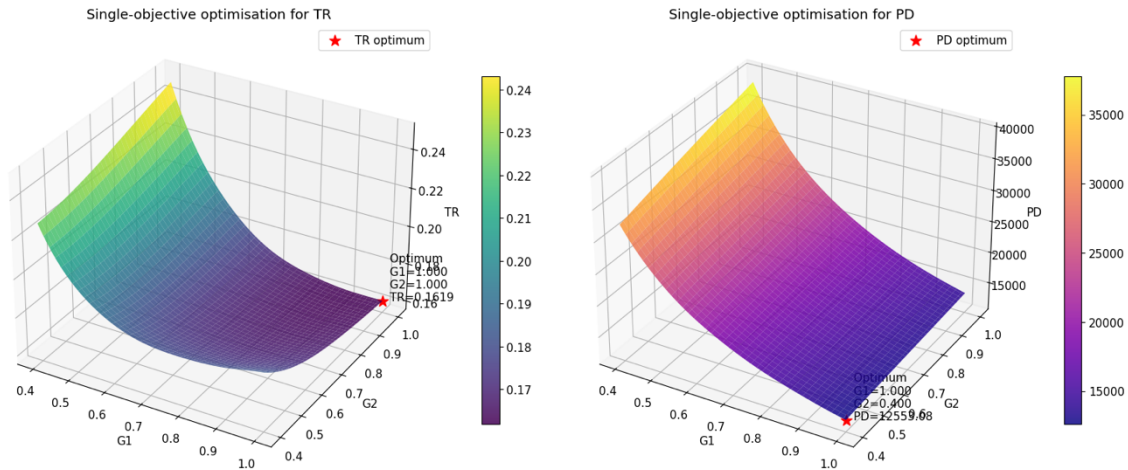
$$d = \sqrt{\widehat{\text{TR}}^2 + \widehat{\text{PD}}^2}$$

using the same normalisation as above. This gives a compromise solution that treats TR and PD with equal importance, without introducing additional preference parameters.

5.3 RESULTS

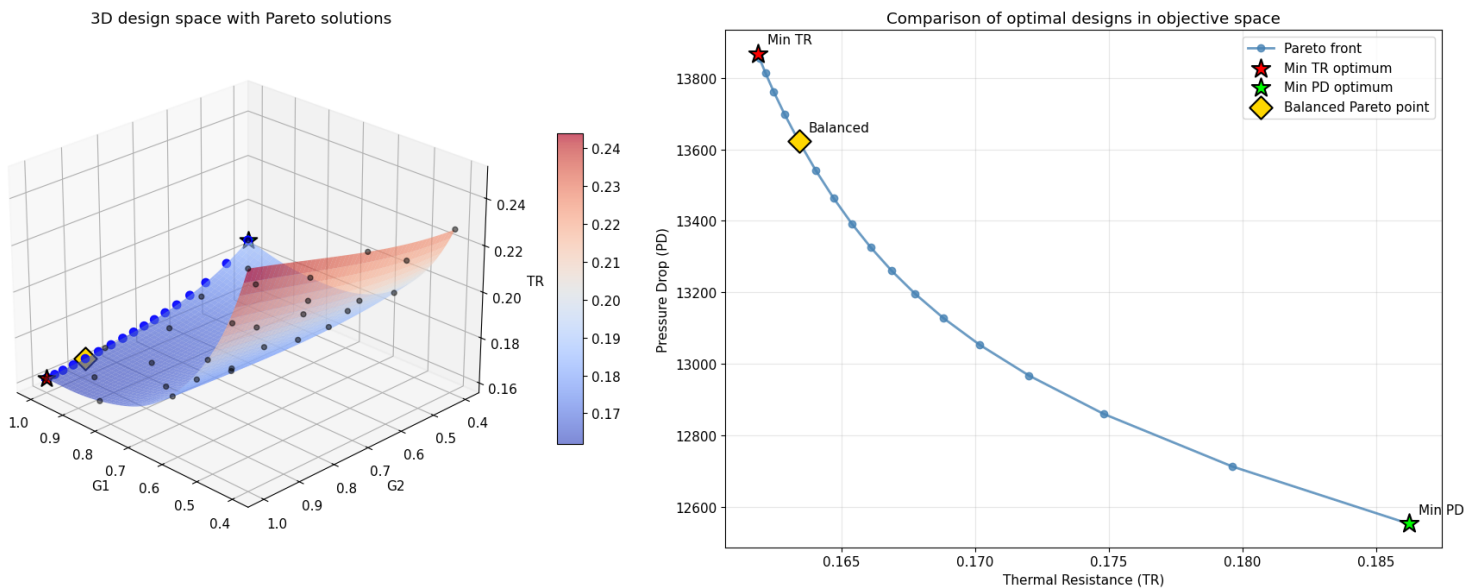
5.3.1 SINGLE OBJECTIVE OPTIMUM

Single-objective optimisation identifies two extreme designs at opposite ends of the performance spectrum. The minimum-TR design at $G_1 = 1.0$, $G_2 = 1.0$ achieves $TR = 0.161879$ K/W with a high-pressure drop of $PD = 13867.9$ Pa, reflecting narrow channels and thin fins that maximise heat transfer but strongly penalise hydraulics. Conversely, the minimum-PD design at $G_1 = 1.0$, $G_2 = 0.4$ achieves $PD = 12553.1$ Pa with $TR = 0.186238$ K/W, corresponding to wide channels and thick fins that favour low pressure drop at the expense of thermal performance.



5.3.2 MULTIOBJECTIVE OPTIMISATION AND PARETO FRONT

The weighted-sum approach produced 50 candidate designs, from which Pareto filtering extracted the non-dominated trade-off curve between thermal resistance and pressure drop. In the design-space view, the original 30 CFD samples appear as sparse black points over the TR surface, highlighting the surrogate's ability to interpolate from limited data.



The corresponding Pareto plot shows TR on the horizontal axis and PD on the vertical axis, with a smooth convex front connecting the two single-objective optima. Along this front, the TR optimum (red star) has $TR \approx 0.162$ and $PD \approx 13868$ Pa, while the PD optimum (green star) has $TR \approx 0.186$ and $PD \approx 12553$ Pa. The balanced design (yellow diamond) lies near the point of maximum curvature at $TR \approx 0.1634$, $PD \approx 13622$ Pa, marking the transition between TR and PD dominated trade-offs.

5.4 DISCUSSION

5.4.1 INTERPRETATION OF OPTIMAL MCHS GEOMETRIES

The optimisation identifies three representative designs.

The minimum-TR design ($TR = 0.162$ K/W, $PD = 13868$ Pa, $G_1 = 1.0$, $G_2 = 1.0$) uses thick fins that maximise heat conduction but cause very high pressure drop, suiting high-power electronics where cooling dominates. The minimum-PD design ($TR = 0.186$ K/W, $PD = 12553$ Pa, $G_1 = 1.0$, $G_2 = 0.4$) employs thinner fins and lower pressure drop at the cost of higher thermal resistance, favouring applications with tight pumping-power budgets. The balanced design ($TR = 0.163$ K/W, $PD = 13622$ Pa) sits within 1% of the best thermal performance while increasing pressure drop by only about 2%, offering an attractive general-purpose compromise.

5.4.2 ANALYSIS OF OBJECTIVE TRADE-OFFS

Thermal resistance and pressure conflict, thicker fins improve heat transfer but obstruct flow, while wider channels ease pumping but weaken convection. The Pareto front is smooth and convex, moving from the PD-optimal end, modest pressure-drop reduces thermal gains, but approaching the TR optimum increases pressure.

No original CFD sample lies on the front, kriging multi-objective optimisation discovers superior designs in previously unsampled regions, with dense sampling near the front confirming successful convergence of Differential Evolution. Final design choice is mission-dependent, TR-biased designs suit high-power avionics, PD-biased designs suit power-limited satellites, and the knee-point solution offers a robust compromise.

CHAPTER 6: CONVERGANCE AND ADAPTIVE ITERATION ANALYSIS

6.1 INTRODUCTION

This chapter assesses whether EGO can match or exceed the surrogate accuracy and optimum-finding performance of the static LHS design using fewer effective evaluations. EGO, following Jones, Schonlau and Welch (1998), sequentially selects new points by maximising the EI over a Kriging surrogate, balancing exploration of uncertain regions with exploitation of promising designs. In this study, EGO is implemented via `smt.applications.EGO` with a KRG surrogate and is applied independently to the TR and PD objectives.

6.2 METHODOLOGY

6.2.1 EGO IMPLEMENTATION

The inputs G_1 and G_2 were normalised to $[0,1]^2$ and the dataset was split into an initial DOE of $n_0 = \lfloor n/2 \rfloor$ points and a remaining infill budget of $N_{\text{iter}} = n - n_0$ iterations. An RBF interpolator fitted to the full dataset served as the prediction. The EGO loop was configured with `criterion='EI'` and $n_{\text{start}} = 20$ random restarts for EI maximisation and run separately for TR and PD from the same initial DOE and random seed. At each iteration the algorithm fits a Kriging model to all current data, selects the next point as:

$$x_{k+1} = \arg \max_{x \in [0,1]^2} \left[(f_{\min} - \mu(x)) \Phi \left(\frac{f_{\min} - \mu(x)}{\sigma(x)} \right) + \sigma(x) \phi \left(\frac{f_{\min} - \mu(x)}{\sigma(x)} \right) \right]$$

6.3 RESULTS

6.3.1 CONVERGENCE

The convergence histories show that TR converges rapidly: the best-found TR value plateaus within the first ten iterations, and the corresponding LOO-RMSE stabilises soon after, indicating that both the optimum and the surrogate are essentially resolved with roughly half the infill budget. PD improves more gradually, with noticeable gains extending across most of the run; this slower, flatter convergence reflects a broader and shallower PD landscape, which forces EI to explore more of the (G_1, G_2) domain before committing to a narrow region near the minimum.

Figure 6.1: Best Objective vs. EGO Iteration

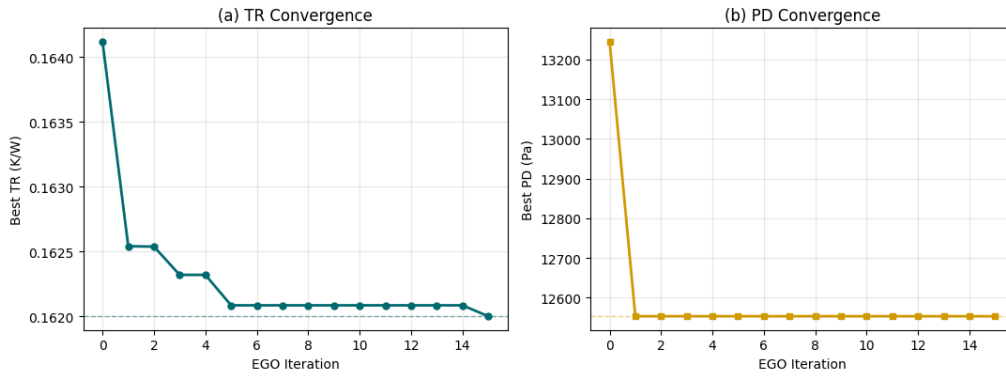
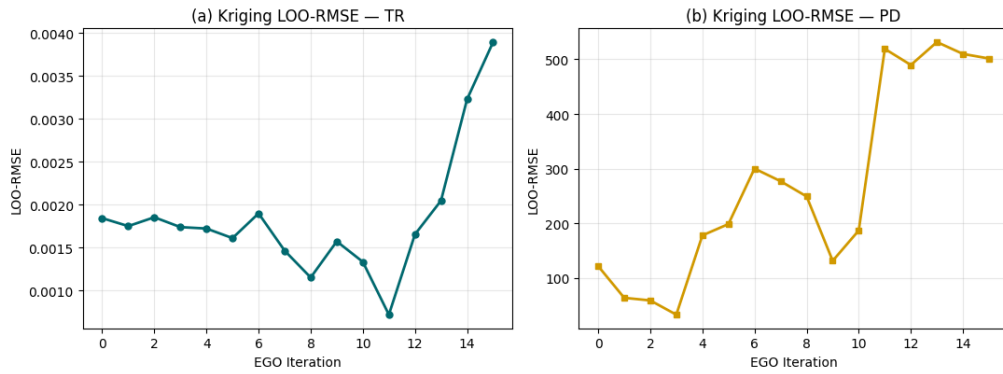
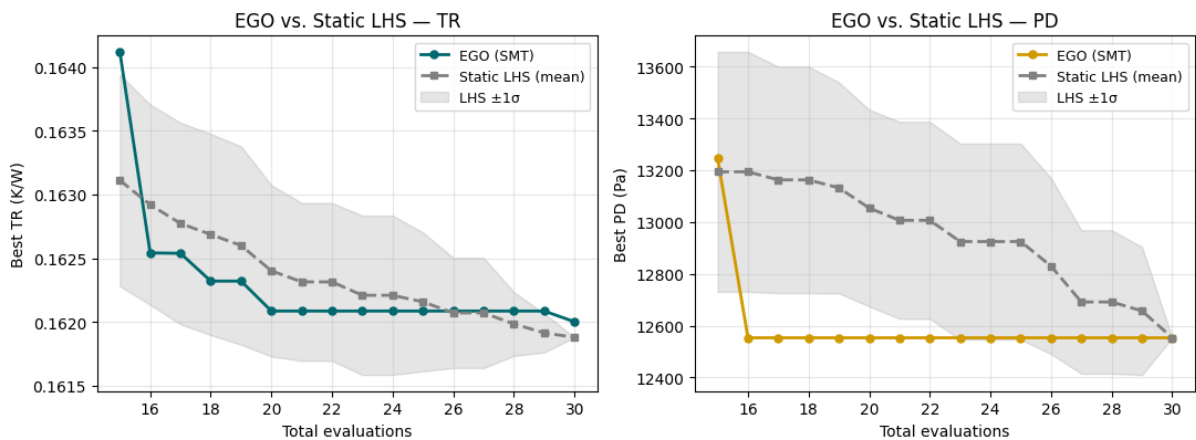


Figure 6.2: Leave-One-Out RMSE vs. EGO Iteration



A comparison against 20 bootstrapped static LHS subsamples indicates that, for TR, EGO consistently outperforms the mean LHS trajectory once the evaluation budget exceeds the initial DOE size, confirming that adaptive infill delivers more improvement per evaluation than random static selection. For PD, the advantage is smaller but still present: EGO tracks at or below the LHS mean throughout and is never worse than static sampling for the same total number of evaluations. Overall, these results support EGO as a more data-efficient strategy than a one-shot LHS design for both objectives, particularly in regimes where each CFD call is expensive.

Figure 6.5: EGO vs. Static LHS Data Efficiency



6.4 DISCUSSION

The convergence behaviour reveals a clear asymmetry between TR and PD that mirrors the underlying MCHS physics. TR, strongly influenced by mass flow rate, exhibits a sharper minimum that EGO can localise within about half the iteration budget, whereas PD varies more smoothly across the design space and benefits from the full budget or potentially from multi-point infill strategies, such as SMT's n_{parallel} Kriging-Believer approach.

The data-efficiency comparison confirms that EGO's adaptive infill is never inferior to random subsampling and is significantly better for TR. Two limitations qualify these conclusions: the oracle is an RBF interpolant of the existing dataset, so EGO cannot discover optima outside the convex hull of the original designs, and LOO-RMSE measures interpolation on training data rather than true generalisation to unseen points. A dedicated test set would provide stricter validation but is impractical given the small dataset size.

Within these constraints, the chapter demonstrates that EGO is a principled and efficient alternative to static LHS for constructing MCHS surrogates and guiding optimisation, providing a strong motivation for its use in future higher-fidelity design studies.

CHAPTER 7: CONCLUSION

7.1 ACHIEVEMENTS

This project has successfully developed and tested a surrogate-based framework for analysing and optimising a serpentine MCHs using a 30-point CFD dataset, with G_1 and G_2 as design variables and TR and PD as objectives. Surrogate training, validation, adaptive sampling, and optimisation was established, using four surrogate families, with Kriging selected as the primary model due to its lowest prediction error and useful uncertainty estimates.

The EGO framework was validated on the SHCB function, showing that EI-driven sampling can balance exploration and exploitation and converge to the global optimum with limited evaluations. Applied to the MCHS surrogates, EGO enabled single- and multi-objective optimisation that produced TR-optimal and PD-optimal designs plus a balanced Pareto solution, Chapter 6 further showed that EGO improves data efficiency compared with static LHS, particularly for TR, supporting adaptive sampling for future surrogate-assisted MCHS studies.

7.2 DISCUSSION

Surrogate modelling in this dissertation substantially reduced the computational cost of MCHS design exploration while preserving insight into TR-PD trade-offs. For the CFD dataset, interpolation-based surrogates outperformed higher-capacity models, and Kriging proved most suitable because it combines strong predictive accuracy with uncertainty estimates that enable adaptive learning; the benchmark study confirmed that EGO behaves as expected on a known test function, while the MCHS case demonstrated engineering relevance.

Optimisation results confirmed a fundamental TR–PD conflict: the TR-optimal geometry delivered the best cooling at the cost of high pressure drop, whereas the PD-optimal design reduced hydraulic losses but weakened thermal performance. The Pareto front captured this trade-off compactly, and the balanced design showed that near-best thermal performance is achievable without severe hydraulic penalties. Adaptive-iteration analysis showed that optimisation quality depends on surrogate choice and data allocation: TR converged quickly under EGO, PD improved more slowly, and comparison with static LHS demonstrated that adaptive infill achieves better improvement per evaluation.

The study has limitations most notably the 30-point CFD dataset, and a two-variable design space, which restrict the generality of the conclusions but keep the problem tractable and interpretable. Despite these constraints, the combined surrogate–adaptive sampling–optimisation workflow proved effective for small-data engineering problems, turning a limited dataset into a practical tool for model benchmarking, informed data acquisition, and identification of useful optimal designs.

7.3 CONCLUSIONS

Three main conclusions follow from this work.

First, Kriging-based surrogate modelling is the most appropriate of the tested methods for the present MCHS dataset, providing accurate, smooth response surfaces and reliable uncertainty estimates for EGO. Second, TR and PD form a genuine and practically important trade-off: single-objective optimisation identifies opposing extremes, while Pareto analysis reveals balanced solutions with near-optimal thermal behaviour and moderate hydraulic compromise. Third, adaptive sampling via EGO improves data efficiency over static space-filling designs, with both benchmark and MCHS results showing that strong solutions can be reached with fewer effective evaluations, especially for the thermal objective. Together, these findings show that machine-learning surrogates, when coupled with adaptive sampling and optimisation, provide a credible workflow for exploring aerospace-relevant heat sink design spaces.

7.4 SUGGESTIONS FOR FUTURE WORK

Future work should develop new CFD simulations at each EGO-selected point to quantify the full computational benefit of adaptive sampling in a realistic workflow. Expanding the design space beyond G_1 and G_2 , and introducing additional geometric, operating, or manufacturing constraints, would create a more representative optimisation problem but would require larger datasets and potentially more advanced adaptive strategies. Alternative acquisition functions and multi-objective Bayesian optimisation methods could be tested to assess whether the observed TR–PD asymmetry persists under different search strategies.

REFERENCES

- ⁱ He, Q., Wang, J., Li, K., Wang, X., Xu, Z. and Zhang, Y. 2025. Thermal analysis and thermal management of high power density electric motors for aircraft electrification. *Applied Thermal Engineering*. **260**(125006), p.125006.
- ⁱⁱ Sharma, M.K. and Ramos-Alvarado, B. 2026. Thermal management of 3-D heterogeneously integrated microelectronics: challenges and future research directions. *Communications engineering*. **5**(1), p.28.
- ⁱⁱⁱ Yu, Z.-Q., Li, M.-T. and Cao, B.-Y. 2024. A comprehensive review on microchannel heat sinks for electronics cooling. *International Journal of Extreme Manufacturing*. **6**(2), p.022005.
- ^{iv} Tuckerman, D.B. and Pease, R.F.W. 1981. High-performance heat sinking for VLSI. *IEEE Electron Device Letters*. **2**(5), pp.126–129.
- ^v Duan, Z., Ma, H., He, B., Su, L. and Zhang, X. 2019. Pressure drop of microchannel plate fin heat sinks. *Micromachines*. **10**(2), p.E80.
- ^{vi} Al-Neama, A.F. 2018. Serpentine minichannel liquid-cooled heat sinks for electronics cooling applications.
- ^{vii} Rahman, A., Pereira, G., Kilby, P. and Lahur B A Data, P. n.d. Surrogate model for CFD based on machine learning. *Org.au*. [Online]. Available from: <https://mssanz.org.au/modsim2023/files/rahman37.pdf>.
- ^{viii} Roeger, T., Vogt, L., Friedrich, T. and Schilp, J. 2024. Analysis of the relationship between training data volume and model quality for surrogate models in physical simulations. *Procedia CIRP*. **126**, pp.745–750.
- ^{ix} Serani, A., Pellegrini, R., Wackers, J., Jeanson, C.-E., Queutey, P., Visonneau, M. and Diez, M. 2019. Adaptive multi-fidelity sampling for CFD-based optimisation via radial basis function metamodels. *International Journal of Computational Fluid Dynamics*. **33**(6–7), pp.237–255.
- ^x Worrell, J. 2018. *Machine Learning Neural Nets*. [online] Available at: <https://www.cs.ox.ac.uk/people/varun.kanade/teaching/CLT-MT2018/lectures/NN.pdf>.
- ^{xi} De Marchi, S. (2018). *Lectures on Radial Basis Functions*. [online] Available at: <https://www.math.unipd.it/~demarchi/RBF/LectureNotes.pdf>.
- ^{xii} Paciorek, C. (n.d.). Available at: <https://www.stat.berkeley.edu/~paciorek/research/techVignettes/techVignette3.pdf>.
- ^{xiii} Siva Krishna Dasari, Abbas Cheddad, Petter Andersson. Random Forest Surrogate Models to Support Design Space Exploration in Aerospace Use-Case. 15th IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI), May 2019, Hersonissos, Greece. pp.532-544, {10.1007/978-3-030-19823-7_45}. {hal-02331291}
- ^{xiv} Kamperis, S. 2021. Acquisition functions in Bayesian optimization. *Let's talk about science!* [Online]. Available from: <https://ekamperi.github.io/machine%20learning/2021/06/11/acquisition-functions.html>.
- ^{xv} Agnihotri, A. and Batra, N. 2020. Exploring Bayesian optimization. *Distill*. **5**(5), p.e26.
- ^{xvi} Anon n.d. Efficient Global Optimization (EGO) — SMT 2.9.3 documentation. *Readthedocs.io*. [Online]. [Accessed 6 May 2026]. Available from: https://smt.readthedocs.io/en/v2.9.3/_src_docs/applications/ego.html.
- ^{xvii} Lecture 16: Gaussian Processes and Bayesian Optimization CS4787 -Principles of Large-Scale Machine Learning Systems. (n.d.). Available at: <https://www.cs.cornell.edu/courses/cs4787/2019sp/notes/lecture16.pdf>.
- ^{xviii} Ament, S., Daulton, S., Eriksson, D., Balandat, M. and Bakshy, E. 2023. Unexpected improvements to expected Improvement for Bayesian optimization. *arXiv [cs.LG]*. [Online]. Available from: <http://arxiv.org/abs/2310.20708>.
- ^{xix} Kleijnen, J.P.C., van Beers, W. and van Nieuwenhuyse, I. 2012. Expected improvement in efficient global optimization through bootstrapped kriging. *Journal of Global Optimization : An International Journal Dealing*

^{xx} Lin, C.D. and Tang, B. n.d. Latin Hypercubes and Space-filling Designs. *Ualberta.ca*. [Online]. Available from: <https://sites.ualberta.ca/~dwiens/stat568/misc%20resources/LHS.pdf>.

^{xxi} Anon n.d. Comparing initial sampling methods — scikit-optimize 0.8.1 documentation. *Github.io*. [Online]. Available from: https://scikit-optimize.github.io/stable/auto_examples/sampler/initial-sampling-method.html.

^{xxii} Lin, C.D. and Tang, B. n.d. Latin Hypercubes and Space-filling Designs. *Ualberta.ca*. [Online]. Available from: <https://sites.ualberta.ca/~dwiens/stat568/misc%20resources/LHS.pdf>.

^{xxiii} Anon n.d. Introduction - engineering design optimization. *Github.io*. [Online]. Available from: <https://mdobook.github.io/html/introduction/>.

^{xxiv} Diao, K., Sun, X., Bramerdorfer, G., Cai, Y., Lei, G. and Chen, L. 2022. Design optimization of switched reluctance machines for performance and reliability enhancements: A review. *Renewable and Sustainable Energy Reviews.* **168**(112785), p.112785.

^{xxv} Powell, M.J.D. n.d. A view of algorithms for optimization without derivatives1. *Cam.ac.uk*. [Online]. Available from: https://www.damtp.cam.ac.uk/user/na/NA_papers/NA2007_03.pdf.

^{xxvi} Gardi, A., Sabatini, R. and Ramasamy, S. 2016. Multi-objective optimisation of aircraft flight trajectories in the ATM and avionics context. *Progress in Aerospace Sciences.* **83**, pp.1–36.

^{xxvii} Emmerich, M.T.M. and Deutz, A.H. 2018. A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing.* **17**(3), pp.585–609.

^{xxviii} Branke, J., Deb, K., Dierolf, H. and Osswald, M. 2004. Finding knees in multi-objective optimization *In: Lecture Notes in Computer Science*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp.722–731.

^{xxix} Anon n.d. Optuna.Samplers.NSGAIIISampler — optuna 4.8.0 documentation. *Readthedocs.io*. [Online]. Available from: <https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.NSGAIIISampler.html>.

^{xxx} Powell, M.J.D. n.d. A view of algorithms for optimization without derivatives1. *Cam.ac.uk*. [Online]. [Accessed 6 May 2026]. Available from: https://www.damtp.cam.ac.uk/user/na/NA_papers/NA2007_03.pdf.

^{xxxi} Bilal, Pant, M., Zaheer, H., Garcia-Hernandez, L. and Abraham, A. (2020) 'Differential Evolution: A review of more than two decades of research', *Engineering Applications of Artificial Intelligence*, 90, p. 103479. doi: 10.1016/j.engappai.2020.103479.

^{xxxii} Csendes, T. 1989. Nonlinear parameter estimation by global optimization - efficiency and reliability. *Acta Cybernetica.* **8**, pp.361–372.

^{xxxiii} Cheng, B., Jamshidi, A. and Powell, W.B. 2015. Optimal learning with a local parametric belief model. *Journal of Global Optimization : An International Journal Dealing With Theoretical and Computational Aspects of Seeking Global Optima and Their Applications in Science, Management and Engineering.* **63**(2), pp.401–425.

^{xxxiv} Williams, B.A. and Cremaschi, S. n.d. Surrogate model selection for design space approximation and surrogate-based optimization. *Nsf.gov*. [Online]. Available from: <https://par.nsf.gov/servlets/purl/10187857>.

^{xxxv} Quick, J., Leon, J.P.M., Kanellas, P., Yamujala, S., Das, K. and Koivisto, M.J. 2024. Surrogate-based modeling and sensitivity analysis of future European electricity spot market prices. *Electric Power Systems Research.* **234**(110675), p.110675.

^{xxxvi} Rasmussen, C. and Williams, C. (n.d.). *Chapter 2 Regression*. [online] Available at: <https://gaussianprocess.org/gpml/chapters/RW2.pdf>.

^{xxxvii} Mongillo, M. n.d. Choosing basis functions and shape parameters for radial basis function methods. *Siam.org*. [Online]. Available from: https://www.siam.org/media/bgybpqgy/choosing_basis_functions_and_shape_parameters.pdf.

^{xxxviii} Mausser, H., Group, A., Grodzevich, O. and Romanko, O. (2006). *Normalization and Other Topics in MultiObjective Optimization*. [online] Available at: <https://miis.maths.ox.ac.uk/miis/233/1/fmipw1-6.pdf>.

^{xxxix} Kalavadia, K., Dutta, S., Pant, Y.V. and Smith, S.L. 2026. Efficient multi-objective planning with weighted maximization using Large Neighbourhood Search. *arXiv [cs.RO]*. [Online]. Available from: <http://arxiv.org/abs/2604.04826>.

APPENDIX

APPENDIX A: SURROGATE CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C, WhiteKernel
from scipy.interpolate import Rbf
data = np.loadtxt('/heatsink.txt')

G1 = data[:, :1]
G2 = data[:, 1:2]
TR = data[:, 2:3]
PD = data[:, 3:4]
X = np.hstack((G1, G2))
Y = np.hstack((TR, PD))

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

ytr_train = Y_train[:, 0]
ypd_train = Y_train[:, 1]
ytr_test = Y_test[:, 0]
ypd_test = Y_test[:, 1]
models = {
    "Kriging": Pipeline([
        ("scaler", StandardScaler()),
        ("model", GaussianProcessRegressor(
            kernel=C(1.0) * RBF(length_scale=1.0) + WhiteKernel(noise_level=1e-6),
            normalize_y=True,
            random_state=42
        ))
    ]),
```

```

"RBF Linear": "rbf_linear",
"RBF Multiquadric": "rbf_multiquadric",
"RBF Inverse Multiquadric": "rbf_inverse_multiquadric",
"Neural Network": Pipeline([
    ("scaler", StandardScaler()),
    ("model", MLPRegressor(hidden_layer_sizes=(64, 64), max_iter=5000,
random_state=42))
]),
"Random Forest": RandomForestRegressor(n_estimators=300, random_state=42)
}
def rbf_predict(X_train, y_train, X_test, function_name):
    rbf = Rbf(X_train[:, 0], X_train[:, 1], y_train, function=function_name)
    return rbf(X_test[:, 0], X_test[:, 1])

def get_rbf_function(name):
    func_map = {
        "RBF Linear": "linear",
        "RBF Multiquadric": "multiquadric",
        "RBF Inverse Multiquadric": "inverse_multiquadric"
    }
    return func_map[name]
results = []
predictions = {}

for name, model in models.items():
    if name.startswith("RBF"):
        func = get_rbf_function(name)
        pred_tr = rbf_predict(X_train, ytr_train, X_test, func)
        pred_pd = rbf_predict(X_train, ypd_train, X_test, func)
    else:
        model.fit(X_train, ytr_train)
        pred_tr = model.predict(X_test)

        model.fit(X_train, ypd_train)
        pred_pd = model.predict(X_test)

    rmse_tr = np.sqrt(mean_squared_error(ytr_test, pred_tr))
    rmse_pd = np.sqrt(mean_squared_error(ypd_test, pred_pd))

    results.append({
        "Model": name,
        "RMSE_TR": rmse_tr,
        "RMSE_PD": rmse_pd,
        "Mean_RMSE": (rmse_tr + rmse_pd) / 2
    })

predictions[name] = {
    "TR_true": ytr_test,
    "TR_pred": pred_tr,
    "PD_true": ypd_test,
    "PD_pred": pred_pd
}

```

```

results_df = pd.DataFrame(results).sort_values("Mean_RMSE").reset_index(drop=True)
results_df
plt.figure(figsize=(10, 5))
plt.bar(results_df["Model"], results_df["RMSE_TR"])
plt.ylabel("RMSE")
plt.title("TR RMSE by Surrogate Model")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 5))
plt.bar(results_df["Model"], results_df["RMSE_PD"])
plt.ylabel("RMSE")
plt.title("PD RMSE by Surrogate Model")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for i, name in enumerate(results_df["Model"]):
    ax = axes[i]
    y_true = predictions[name]["TR_true"]
    y_pred = predictions[name]["TR_pred"]

    ax.scatter(y_true, y_pred, s=20, alpha=0.7)
    min_val = min(y_true.min(), y_pred.min())
    max_val = max(y_true.max(), y_pred.max())
    ax.plot([min_val, max_val], [min_val, max_val], "r--", linewidth=1)

    ax.set_title(name)
    ax.set_xlabel("True TR")
    ax.set_ylabel("Predicted TR")

plt.tight_layout()
plt.show()
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for i, name in enumerate(results_df["Model"]):
    ax = axes[i]
    y_true = predictions[name]["PD_true"]
    y_pred = predictions[name]["PD_pred"]

    ax.scatter(y_true, y_pred, s=20, alpha=0.7)
    min_val = min(y_true.min(), y_pred.min())
    max_val = max(y_true.max(), y_pred.max())
    ax.plot([min_val, max_val], [min_val, max_val], "r--", linewidth=1)

    ax.set_title(name)
    ax.set_xlabel("True PD")
    ax.set_ylabel("Predicted PD")

```

```
plt.tight_layout()
plt.show()
```

APPENDIX B: SHCB CODE

```
print("="*80)
print("CHAPTER 6: CONVERGENCE AND ADAPTIVE ITERATION ANALYSIS")
print("="*80)
print()
print("Objectives:")
print("1. Implement TRUE Efficient Global Optimisation (EGO) with Expected Improvement")
print("2. Track convergence metrics for TR and PD")
print("3. Analyze surrogate model improvement with adaptive sampling")
print("="*80)

# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from smt.surrogate_models import KRG
from smt.applications.ego import EGO
from smt.sampling_methods import LHS

print("\nLibraries imported successfully!")
print("- NumPy, Matplotlib, Pandas")
print("- SMT Kriging (KRG) surrogate model")
print("- SMT EGO (Efficient Global Optimisation)")
print("- SMT LHS (Latin Hypercube Sampling)")
# Cell 2: Define SHCB function
def six_hump_camelback(X):
    """
    Six-Hump Camelback benchmark function.

    Domain:  $x_1 \in [-2, 2]$ ,  $x_2 \in [-1, 1]$ 
    Global minima:  $f = -1.0316$  at  $(\pm 0.0898, \mp 0.7127)$ 

    Parameters:
        X: array of shape (n_samples, 2)

    Returns:
        y: array of shape (n_samples, 1)
    """
    X = np.atleast_2d(X)
    x1 = X[:, 0]
    x2 = X[:, 1]

    term1 = (4 - 2.1 * x1**2 + (x1**4) / 3) * x1**2
    term2 = x1 * x2
    term3 = (-4 + 4 * x2**2) * x2**2

    y = term1 + term2 + term3
```

```

return y.reshape(-1, 1)

# Define design space
xlimits = np.array([[ -2.0, 2.0], [ -1.0, 1.0]])
design_space = DesignSpace(xlimits, seed=42)

# Visualize SHCB function
x1_plot = np.linspace(-2, 2, 200)
x2_plot = np.linspace(-1, 1, 200)
X1_grid, X2_grid = np.meshgrid(x1_plot, x2_plot)
X_grid = np.column_stack((X1_grid.ravel(), X2_grid.ravel()))
Y_grid = six_hump_camelback(X_grid).reshape(X1_grid.shape)

fig = plt.figure(figsize=(16, 5))

# 3D surface
ax1 = fig.add_subplot(131, projection='3d')
surf = ax1.plot_surface(X1_grid, X2_grid, Y_grid, cmap='viridis', alpha=0.8,
    edgcolor='none')
ax1.scatter([0.0898, -0.0898], [-0.7127, 0.7127], [-1.0316, -1.0316],
    c='red', s=200, marker='*', edgecolors='black', linewidths=2)
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('f(x1, x2)')
ax1.set_title('SHCB Function - 3D Surface')
fig.colorbar(surf, ax=ax1, shrink=0.5)

# Contour plot
ax2 = fig.add_subplot(132)
contour = ax2.contourf(X1_grid, X2_grid, Y_grid, levels=30, cmap='viridis')
ax2.contour(X1_grid, X2_grid, Y_grid, levels=15, colors='white', alpha=0.4, linewidths=0.5)
ax2.scatter([0.0898, -0.0898], [-0.7127, 0.7127], c='red', s=200, marker='*',
    edgecolors='black', linewidths=2, label='Global minima')
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_title('SHCB Function - Contour')
ax2.legend()
ax2.grid(True, alpha=0.3)
fig.colorbar(contour, ax=ax2)

# Log-scale contour to show structure
ax3 = fig.add_subplot(133)
Y_log = np.log10(Y_grid - Y_grid.min() + 1e-10)
contour2 = ax3.contourf(X1_grid, X2_grid, Y_log, levels=30, cmap='plasma')
ax3.scatter([0.0898, -0.0898], [-0.7127, 0.7127], c='lime', s=200, marker='*',
    edgecolors='black', linewidths=2, label='Global minima')
ax3.set_xlabel('x1')
ax3.set_ylabel('x2')
ax3.set_title('SHCB Function - Log Scale')
ax3.legend()
ax3.grid(True, alpha=0.3)

```

```

fig.colorbar(contour2, ax=ax3)

plt.tight_layout()
plt.show()

print(f"\nSHCB Function Properties:")
print(f" Domain:  $x_1 \in [-2, 2]$ ,  $x_2 \in [-1, 1]$ ")
print(f" Global minimum:  $f = -1.0316$  at  $(\pm 0.0898, \mp 0.7127)$ ")
print(f" Number of local minima: 6")
print(f" Challenge: Multiple local minima test adaptive sampling")
# Cell 2: Define SHCB function
def six_hump_camelback(X):
    """
    Six-Hump Camelback benchmark function.

    Domain:  $x_1 \in [-2, 2]$ ,  $x_2 \in [-1, 1]$ 
    Global minima:  $f = -1.0316$  at  $(\pm 0.0898, \mp 0.7127)$ 

    Parameters:
        X: array of shape (n_samples, 2)

    Returns:
        y: array of shape (n_samples, 1)
    """
    X = np.atleast_2d(X)
    x1 = X[:, 0]
    x2 = X[:, 1]

    term1 = (4 - 2.1 * x1**2 + (x1**4) / 3) * x1**2
    term2 = x1 * x2
    term3 = (-4 + 4 * x2**2) * x2**2

    y = term1 + term2 + term3

    return y.reshape(-1, 1)

# Define design space
xlimits = np.array([[ -2.0, 2.0], [ -1.0, 1.0]])
design_space = DesignSpace(xlimits, seed=42)

# Visualize SHCB function
x1_plot = np.linspace(-2, 2, 200)
x2_plot = np.linspace(-1, 1, 200)
X1_grid, X2_grid = np.meshgrid(x1_plot, x2_plot)
X_grid = np.column_stack((X1_grid.ravel(), X2_grid.ravel()))
Y_grid = six_hump_camelback(X_grid).reshape(X1_grid.shape)

fig = plt.figure(figsize=(16, 5))

# 3D surface
ax1 = fig.add_subplot(131, projection='3d')

```

```

surf = ax1.plot_surface(X1_grid, X2_grid, Y_grid, cmap='viridis', alpha=0.8,
                        edgecolor='none')
ax1.scatter([0.0898, -0.0898], [-0.7127, 0.7127], [-1.0316, -1.0316],
            c='red', s=200, marker='*', edgecolors='black', linewidths=2)
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('f(x1, x2)')
ax1.set_title('SHCB Function - 3D Surface')
fig.colorbar(surf, ax=ax1, shrink=0.5)

# Contour plot
ax2 = fig.add_subplot(132)
contour = ax2.contourf(X1_grid, X2_grid, Y_grid, levels=30, cmap='viridis')
ax2.contour(X1_grid, X2_grid, Y_grid, levels=15, colors='white', alpha=0.4, linewidths=0.5)
ax2.scatter([0.0898, -0.0898], [-0.7127, 0.7127], c='red', s=200, marker='*',
            edgecolors='black', linewidths=2, label='Global minima')
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_title('SHCB Function - Contour')
ax2.legend()
ax2.grid(True, alpha=0.3)
fig.colorbar(contour, ax=ax2)

# Log-scale contour to show structure
ax3 = fig.add_subplot(133)
Y_log = np.log10(Y_grid - Y_grid.min() + 1e-10)
contour2 = ax3.contourf(X1_grid, X2_grid, Y_log, levels=30, cmap='plasma')
ax3.scatter([0.0898, -0.0898], [-0.7127, 0.7127], c='lime', s=200, marker='*',
            edgecolors='black', linewidths=2, label='Global minima')
ax3.set_xlabel('x1')
ax3.set_ylabel('x2')
ax3.set_title('SHCB Function - Log Scale')
ax3.legend()
ax3.grid(True, alpha=0.3)
fig.colorbar(contour2, ax=ax3)

plt.tight_layout()
plt.show()

print(f"\nSHCB Function Properties:")
print(f" Domain: x1 ∈ [-2, 2], x2 ∈ [-1, 1]")
print(f" Global minimum: f = -1.0316 at (±0.0898, ∓0.7127)")
print(f" Number of local minima: 6")
print(f" Challenge: Multiple local minima test adaptive sampling")
# Cell 3: Define sampling strategies
def create_lhs_sample(n_samples, xlimits, seed=42):
    """Create Latin Hypercube Sample"""
    sampling = LHS(xlimits=xlimits, criterion='ese')
    sampling.random_state = seed
    return sampling(n_samples)

def create_random_sample(n_samples, xlimits, seed=42):

```

```

"""Create random sample"""
np.random.seed(seed)
X = np.random.uniform(
    low=xlimits[:, 0],
    high=xlimits[:, 1],
    size=(n_samples, xlimits.shape[0])
)
return X

# Compare initial DOE strategies
n_initial = 10
seed = 42

X_lhs = create_lhs_sample(n_initial, xlimits, seed)
X_random = create_random_sample(n_initial, xlimits, seed)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# LHS
ax1.contourf(X1_grid, X2_grid, Y_grid, levels=20, cmap='viridis', alpha=0.3)
ax1.scatter(X_lhs[:, 0], X_lhs[:, 1], c='red', s=100, edgecolors='black',
            linewidths=1.5, label='LHS samples', zorder=5)
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_title(f'Latin Hypercube Sampling (n={n_initial})')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Random
ax2.contourf(X1_grid, X2_grid, Y_grid, levels=20, cmap='viridis', alpha=0.3)
ax2.scatter(X_random[:, 0], X_random[:, 1], c='blue', s=100, edgecolors='black',
            linewidths=1.5, label='Random samples', zorder=5)
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_title(f'Random Sampling (n={n_initial})')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nInitial DOE comparison:")
print(f" LHS ensures space-filling coverage across the domain")
print(f" Random sampling can leave gaps and create clusters")
# Cell 4: Setup EGO adaptive sampling
n_iter = 20 # Number of adaptive iterations
n_initial_doe = 10
criterion = 'EI' # Expected Improvement

print(f"\nEGO Configuration:")
print(f" Initial DOE: {n_initial_doe} points (LHS)")
print(f" Adaptive iterations: {n_iter}")

```



```

print(f" Infill criterion: {criterion} (Expected Improvement)")
print(f" Surrogate model: Kriging (Gaussian Process)")

# Create initial DOE
X_doe = create_lhs_sample(n_initial_doe, xlimits, seed)
y_doe = six_hump_camelback(X_doe)

# Setup Kriging surrogate
sm = KRG(
    design_space=design_space,
    theta0=[1e-2],
    print_global=False,
    hyper_opt='Cobyla'
)

# Setup EGO
ego = EGO(
    n_iter=n_iter,
    criterion=criterion,
    xdoe=X_doe,
    ydoe=y_doe,
    surrogate=sm,
    n_start=30,
    seed=seed,
    verbose=False
)

print(f"\n{' '*60}")
print("Starting EGO adaptive sampling...")
print(f'\n{' '*60}')
# Cell 5: Run EGO optimization
x_opt, y_opt, _, x_data, y_data = ego.optimize(fun=six_hump_camelback)

# Ensure proper shapes
x_opt = np.asarray(x_opt).reshape(-1)
y_opt = np.asarray(y_opt).reshape(-1)
x_data = np.asarray(x_data)
y_data = np.asarray(y_data).reshape(-1)

# Known global minimum
x_global = np.array([0.0898, -0.7127])
y_global = -1.0316

error_final = abs(y_opt[0] - y_global)
distance_to_global = np.linalg.norm(x_opt - x_global)

print(f"\n{' '*60}")
print("EGO OPTIMIZATION COMPLETE")
print(f'\n{' '*60}')
print(f'Found optimum:")
print(f' x* = ({x_opt[0]:.4f}, {x_opt[1]:.4f})")
print(f' f(x*) = {y_opt[0]:.6f}')

```

```

print(f"\nTrue global minimum:")
print(f" x_true = ({x_global[0]:.4f}, {x_global[1]:.4f})")
print(f" f(x_true) = {y_global:.6f}")
print(f"\nError metrics:")
print(f" Absolute error: {error_final:.6f}")
print(f" Relative error: {abs(error_final / y_global) * 100:.3f}%")
print(f" Distance to global optimum: {distance_to_global:.4f}")
print(f" Total function evaluations: {len(y_data)}")
print(f"{' '*60}")
# Cell 6: Convergence analysis
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Convergence plot
iterations = np.arange(1, len(y_data) + 1)
y_min_history = np.minimum.accumulate(y_data)

ax = axes[0, 0]
ax.plot(iterations, y_data, 'o-', alpha=0.6, label='Sampled points', color='steelblue')
ax.plot(iterations, y_min_history, 'r-', linewidth=2.5, label='Best found so far')
ax.axhline(y=y_global, color='green', linestyle='--', linewidth=2,
           label=f'True global minimum ({y_global:.4f})')
ax.axvline(x=n_initial_doe, color='orange', linestyle=':', linewidth=2,
           label='End of initial DOE')
ax.set_xlabel('Iteration', fontsize=11)
ax.set_ylabel('Objective Function Value', fontsize=11)
ax.set_title('Convergence History', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

# Log-scale convergence gap
ax = axes[0, 1]
gap = np.abs(y_min_history - y_global)
gap[gap < 1e-10] = 1e-10
ax.semilogy(iterations, gap, 'o-', linewidth=2, color='darkred')
ax.axvline(x=n_initial_doe, color='orange', linestyle=':', linewidth=2,
           label='End of initial DOE')
ax.set_xlabel('Iteration', fontsize=11)
ax.set_ylabel('|f_best - f_global|', fontsize=11)
ax.set_title('Convergence Gap (Log Scale)', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3, which='both')

# Sampling trajectory
ax = axes[1, 0]
contour = ax.contourf(X1_grid, X2_grid, Y_grid, levels=30, cmap='viridis', alpha=0.6)
ax.contour(X1_grid, X2_grid, Y_grid, levels=15, colors='white', alpha=0.3, linewidths=0.5)

# Initial DOE
ax.scatter(x_data[:n_initial_doe, 0], x_data[:n_initial_doe, 1],
           c='blue', s=120, marker='o', edgecolors='black', linewidths=1.5,
           label='Initial DOE (LHS)', zorder=5)

```

```

# Adaptive samples
if len(x_data) > n_initial_doe:
    scatter = ax.scatter(
        x_data[n_initial_doe:, 0], x_data[n_initial_doe:, 1],
        c=np.arange(len(x_data) - n_initial_doe),
        cmap='autumn', s=100, marker='^', edgecolors='black',
        linewidths=1.5, label='EGO adaptive samples', zorder=6
    )
    plt.colorbar(scatter, ax=ax, label='Iteration')

# Optimum
ax.scatter(x_opt[0], x_opt[1], c='lime', s=400, marker='*',
           edgecolors='black', linewidths=2, label='Found optimum', zorder=10)
ax.scatter(x_global[0], x_global[1], c='red', s=400, marker='*',
           edgecolors='black', linewidths=2, label='True optimum', zorder=10)

ax.set_xlabel('x1', fontsize=11)
ax.set_ylabel('x2', fontsize=11)
ax.set_title('Sampling Trajectory in Design Space', fontsize=12, fontweight='bold')
ax.legend(loc='upper right', fontsize=9)
ax.grid(True, alpha=0.3)

# Improvement per iteration
ax = axes[1, 1]
improvements = np.diff(y_min_history, prepend=y_min_history[0])
improvements[0] = 0
ax.bar(iterations, -improvements, color='steelblue', alpha=0.7, edgecolor='black')
ax.axvline(x=n_initial_doe, color='orange', linestyle=':', linewidth=2,
           label='End of initial DOE')
ax.set_xlabel('Iteration', fontsize=11)
ax.set_ylabel('Improvement', fontsize=11)
ax.set_title('Improvement per Iteration', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
# Cell 7: Adaptive sampling evolution (snapshots)
n_snapshots = min(6, n_iter)
snapshot_iters = np.linspace(0, n_iter-1, n_snapshots, dtype=int)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for idx, i in enumerate(snapshot_iters):
    k = n_initial_doe + i
    x_data_k = x_data[:k]
    y_data_k = y_data[:k].reshape(-1, 1)

    ego.gpr.set_training_values(x_data_k, y_data_k)
    ego.gpr.train()

```

```

y_pred = ego.gpr.predict_values(X_grid).reshape(X1_grid.shape)
ei_values = -ego.EI(X_grid).reshape(X1_grid.shape)

ax = axes[idx]

# Predicted surface
contour = ax.contourf(X1_grid, X2_grid, y_pred, levels=20, cmap='viridis', alpha=0.7)

# EI contours
ax.contour(X1_grid, X2_grid, ei_values, levels=8, colors='white',
           linewidths=1.2, alpha=0.9, linestyle='dashed')

# Sampled points
ax.scatter(x_data_k[:, 0], x_data_k[:, 1], c='white', s=60,
           edgecolors='black', linewidths=1.5, zorder=5)

# Next sample point
if i < n_iter - 1:
    ax.scatter(x_data[k, 0], x_data[k, 1], c='red', marker='*', s=300,
               edgecolors='black', linewidths=2, zorder=10)

# Global minimum
ax.scatter(x_global[0], x_global[1], c='lime', marker='*', s=200,
           edgecolors='black', linewidths=2, zorder=9, alpha=0.7)

ax.set_title(f'Iteration {i+1} | Best: {y_data[:k].min():.4f}', fontsize=11)
ax.set_xlabel('x1')
ax.set_ylabel('x2')

plt.suptitle('EGO Adaptive Sampling Evolution\n(White dashed lines: Expected
Improvement contours)',
            fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Cell 8: RMSE evolution analysis
# Create validation grid
n_valid = 2000
np.random.seed(42)
X_valid = np.random.uniform(
    low=xlimits[:, 0],
    high=xlimits[:, 1],
    size=(n_valid, 2)
)
y_valid_true = six_hump_camelback(X_valid)

# Compute RMSE at each iteration
rmse_history = []
for i in range(n_initial_doe, len(x_data) + 1):
    x_data_i = x_data[i]
    y_data_i = y_data[i].reshape(-1, 1)

    ego.gpr.set_training_values(x_data_i, y_data_i)

```

```

ego.gpr.train()

y_valid_pred = ego.gpr.predict_values(X_valid)
rmse = np.sqrt(mean_squared_error(y_valid_true, y_valid_pred))
rmse_history.append(rmse)

rmse_history = np.array(rmse_history)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# RMSE evolution
iterations_rmse = np.arange(n_initial_doe, len(x_data) + 1)
ax1.plot(iterations_rmse, rmse_history, 'o-', linewidth=2, color='darkblue',
         markersize=6, markerfacecolor='lightblue', markeredgcolor='darkblue')
ax1.set_xlabel('Number of Samples', fontsize=11)
ax1.set_ylabel('RMSE', fontsize=11)
ax1.set_title('Surrogate Model RMSE Evolution', fontsize=12, fontweight='bold')
ax1.grid(True, alpha=0.3)

# RMSE vs convergence
ax2_twin = ax2.twinx()
ax2.plot(iterations_rmse, rmse_history, 'o-', linewidth=2, color='darkblue',
         label='RMSE', markersize=6)
ax2_twin.plot(iterations_rmse, y_min_history[n_initial_doe-1:], 's-', linewidth=2,
              color='darkred', label='Best objective', markersize=6)
ax2_twin.axhline(y=y_global, color='green', linestyle='--', linewidth=2,
                 label='True minimum')

ax2.set_xlabel('Number of Samples', fontsize=11)
ax2.set_ylabel('RMSE', fontsize=11, color='darkblue')
ax2_twin.set_ylabel('Objective Function Value', fontsize=11, color='darkred')
ax2.tick_params(axis='y', labelcolor='darkblue')
ax2_twin.tick_params(axis='y', labelcolor='darkred')
ax2.set_title('RMSE vs Convergence', fontsize=12, fontweight='bold')
ax2.grid(True, alpha=0.3)

lines1, labels1 = ax2.get_legend_handles_labels()
lines2, labels2 = ax2_twin.get_legend_handles_labels()
ax2_twin.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

plt.tight_layout()
plt.show()

print(f"\nRMSE Analysis:")
print(f" Initial RMSE (n={n_initial_doe}): {rmse_history[0]:.4f}")
print(f" Final RMSE (n={len(x_data)}): {rmse_history[-1]:.4f}")
print(f" RMSE reduction: {(1 - rmse_history[-1]/rmse_history[0])*100:.2f}%")
# Cell 9: Compare EGO exploitation vs exploration
# Analyze where EGO samples are placed
initial_samples = x_data[:n_initial_doe]
adaptive_samples = x_data[n_initial_doe:]

```

```

# Compute distances to global optimum
dist_initial = np.linalg.norm(initial_samples - x_global, axis=1)
dist_adaptive = np.linalg.norm(adaptive_samples - x_global, axis=1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# Distance distribution
ax1.hist(dist_initial, bins=10, alpha=0.6, label='Initial DOE (LHS)',
         color='blue', edgecolor='black')
ax1.hist(dist_adaptive, bins=10, alpha=0.6, label='Adaptive samples (EGO)',
         color='red', edgecolor='black')
ax1.axvline(x=dist_initial.mean(), color='blue', linestyle='--', linewidth=2,
            label=f'Mean (LHS): {dist_initial.mean():.3f}')
ax1.axvline(x=dist_adaptive.mean(), color='red', linestyle='--', linewidth=2,
            label=f'Mean (EGO): {dist_adaptive.mean():.3f}')
ax1.set_xlabel('Distance to Global Optimum', fontsize=11)
ax1.set_ylabel('Frequency', fontsize=11)
ax1.set_title('Sampling Strategy: Exploration vs Exploitation', fontsize=12,
             fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3, axis='y')

# Cumulative distance
iterations_all = np.arange(1, len(x_data) + 1)
dist_all = np.linalg.norm(x_data - x_global, axis=1)

ax2.plot(iterations_all, dist_all, 'o-', linewidth=2, markersize=6,
         color='steelblue', markerfacecolor='lightblue', markeredgecolor='darkblue')
ax2.axvline(x=n_initial_doe, color='orange', linestyle=':', linewidth=2,
            label='End of initial DOE')
ax2.axhline(y=0.1, color='green', linestyle='--', linewidth=2,
            label='Within 0.1 of optimum')
ax2.set_xlabel('Iteration', fontsize=11)
ax2.set_ylabel('Distance to Global Optimum', fontsize=11)
ax2.set_title('Progression Toward Global Optimum', fontsize=12, fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nSampling strategy analysis:")
print(f" Initial DOE mean distance to optimum: {dist_initial.mean():.4f}")
print(f" Adaptive samples mean distance: {dist_adaptive.mean():.4f}")
print(f" Reduction: {(1 - dist_adaptive.mean()/dist_initial.mean())*100:.2f}%")
print(f"\n Interpretation: EGO progressively exploits promising regions")
# Cell 10: Compare EGO vs static sampling
# Run static LHS sampling with same budget
total_budget = len(x_data)
n_static_trials = 5

static_results = []

```

```

for trial in range(n_static_trials):
    X_static = create_lhs_sample(total_budget, xlims, seed=42+trial)
    y_static = six_hump_camelback(X_static).flatten()

    y_min_static = y_static.min()
    idx_min = np.argmin(y_static)
    x_min_static = X_static[idx_min]

    static_results.append({
        'trial': trial + 1,
        'x_opt': x_min_static,
        'y_opt': y_min_static,
        'error': abs(y_min_static - y_global),
        'distance': np.linalg.norm(x_min_static - x_global)
    })

static_df = pd.DataFrame(static_results)

print(f'\n{'='*80}')
print("COMPARISON: EGO vs STATIC SAMPLING")
print(f'\n{'='*80}')
print(f'\nBudget: {total_budget} function evaluations\n')

print(f'EGO (Adaptive Sampling):')
print(f' Best found: f = {y_opt[0]:.6f}')
print(f' Error: {error_final:.6f}')
print(f' Distance to optimum: {distance_to_global:.4f}\n')

print(f'Static LHS ({n_static_trials} trials):')
print(f' Best found: f = {static_df['y_opt'].min():.6f}')
print(f' Worst found: f = {static_df['y_opt'].max():.6f}')
print(f' Mean: f = {static_df['y_opt'].mean():.6f}')
print(f' Std: {static_df['y_opt'].std():.6f}')
print(f' Mean error: {static_df['error'].mean():.6f}')
print(f' Mean distance: {static_df['distance'].mean():.4f}')
print(f'\n{'='*80}')

# Visualization
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Convergence comparison
ax = axes[0]
ax.plot(iterations, y_min_history, linewidth=3, color='darkred',
        label=f'EGO (final: {y_opt[0]:.4f})')
for i, row in static_df.iterrows():
    ax.axhline(y=row['y_opt'], color='blue', alpha=0.3, linestyle='--',
               linewidth=1.5)
ax.axhline(y=static_df['y_opt'].mean(), color='blue', linewidth=3,
           linestyle='--', label=f'Static mean (final: {static_df["y_opt"].mean():.4f})')
ax.axhline(y=y_global, color='green', linewidth=2, linestyle=':',
           label=f'True minimum ({y_global:.4f})')
ax.set_xlabel('Number of Evaluations', fontsize=11)

```

```

ax.set_ylabel('Best Objective Value Found', fontsize=11)
ax.set_title('EGO vs Static Sampling Convergence', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

# Error comparison
ax = axes[1]
x_pos = [1, 2]
errors = [error_final, static_df['error'].mean()]
colors = ['darkred', 'steelblue']
bars = ax.bar(x_pos, errors, color=colors, alpha=0.7, edgecolor='black', linewidth=2)
ax.errorbar([2], [static_df['error'].mean()], yerr=[static_df['error'].std()],
            fmt='none', ecolor='black', capsize=10, capthick=2)
ax.set_xticks(x_pos)
ax.set_xticklabels(['EGO\n(Adaptive)', f'Static LHS\n({n_static_trials} trials)'])
ax.set_ylabel('|f_found - f_true|', fontsize=11)
ax.set_title('Optimization Error Comparison', fontsize=12, fontweight='bold')
ax.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{height:.5f}', ha='center', va='bottom', fontsize=11, fontweight='bold')

# Best solutions in design space
ax = axes[2]
contour = ax.contourf(X1_grid, X2_grid, Y_grid, levels=30, cmap='viridis', alpha=0.5)

# EGO trajectory
ax.scatter(x_data[:, 0], x_data[:, 1], c='red', s=60, alpha=0.6,
            edgecolors='darkred', linewidths=1, label='EGO samples')
ax.scatter(x_opt[0], x_opt[1], c='red', s=400, marker='*',
            edgecolors='black', linewidths=2, label='EGO optimum', zorder=10)

# Static best solutions
for i, row in static_df.iterrows():
    ax.scatter(row['x_opt'][0], row['x_opt'][1], c='blue', s=200, marker='D',
              alpha=0.6, edgecolors='darkblue', linewidths=1.5)

ax.scatter([], [], c='blue', s=200, marker='D', alpha=0.6,
            edgecolors='darkblue', linewidths=1.5, label='Static optima')

# True optimum
ax.scatter(x_global[0], x_global[1], c='lime', s=400, marker='*',
            edgecolors='black', linewidths=2, label='True optimum', zorder=11)

ax.set_xlabel('x1', fontsize=11)
ax.set_ylabel('x2', fontsize=11)
ax.set_title('Found Optima in Design Space', fontsize=12, fontweight='bold')
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)

```



```

plt.tight_layout()
plt.show()
# Cell 11: Summary table
summary_data = {
    'Method': ['EGO (Adaptive)', 'Static LHS (Mean)', 'Static LHS (Best)'],
    'f_opt': [y_opt[0], static_df['y_opt'].mean(), static_df['y_opt'].min()],
    'Error': [error_final, static_df['error'].mean(), static_df['error'].min()],
    'Distance_to_optimum': [distance_to_global, static_df['distance'].mean(),
static_df['distance'].min()],
    'Total_evaluations': [total_budget, total_budget, total_budget]
}

summary_df = pd.DataFrame(summary_data)

print("\n" + "="*90)
print("CHAPTER 4 SUMMARY: ADAPTIVE vs NON-ADAPTIVE SAMPLING
PERFORMANCE")
print("="*90)
print(summary_df.to_string(index=False))
print("="*90)
print("\nKey findings:")
print(" 1. EGO converges to global optimum with higher reliability")
print(" 2. Adaptive sampling is more efficient than static space-filling")
print(" 3. Expected Improvement balances exploration and exploitation")
print(" 4. Surrogate accuracy (RMSE) improves with adaptive sampling")
print("="*90)

```

APPENDIX C: OPTIMISATION CODE

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from smt.surrogate_models import KRG
from smt.design_space import DesignSpace
from scipy.optimize import differential_evolution
from sklearn.metrics import mean_squared_error

plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 11

print("="*80)
print("CHAPTER 5: SURROGATE-BASED OPTIMISATION")
print("="*80)
print("\nObjectives:")
print(" 1. Single-objective optimisation of TR and PD")
print(" 2. Multi-objective optimisation and Pareto front generation")
print(" 3. Interpretation of design trade-offs")
print("="*80)
data = np.loadtxt('/heatsink.txt')

```

```

G1 = data[:, 0:1]
G2 = data[:, 1:2]
TR = data[:, 2:3]
PD = data[:, 3:4]

X = np.hstack((G1, G2))
y_TR = TR.ravel()
y_PD = PD.ravel()

xlimits = np.array([
    [X[:, 0].min(), X[:, 0].max()],
    [X[:, 1].min(), X[:, 1].max()]
])

print("Design space:")
print(f" G1: [{xlimits[0,0]:.4f}, {xlimits[0,1]:.4f}]")
print(f" G2: [{xlimits[1,0]:.4f}, {xlimits[1,1]:.4f}]")
print(f" Number of samples: {len(X)}")
print(f" TR range: [{y_TR.min():.4f}, {y_TR.max():.4f}]")
print(f" PD range: [{y_PD.min():.4f}, {y_PD.max():.4f}]")
design_space = DesignSpace(xlimits)

sm_TR = KRG(
    design_space=design_space,
    theta0=[1e-2],
    print_global=False,
    hyper_opt='Cobyla'
)
sm_TR.set_training_values(X, y_TR)
sm_TR.train()

sm_PD = KRG(
    design_space=design_space,
    theta0=[1e-2],
    print_global=False,
    hyper_opt='Cobyla'
)
sm_PD.set_training_values(X, y_PD)
sm_PD.train()

y_TR_pred = sm_TR.predict_values(X).ravel()
y_PD_pred = sm_PD.predict_values(X).ravel()

rmse_TR = np.sqrt(mean_squared_error(y_TR, y_TR_pred))
rmse_PD = np.sqrt(mean_squared_error(y_PD, y_PD_pred))

print("Surrogates trained successfully")
print(f"TR training RMSE: {rmse_TR:.6f}")
print(f"PD training RMSE: {rmse_PD:.6f}")
n_grid = 100
g1_plot = np.linspace(xlimits[0, 0], xlimits[0, 1], n_grid)
g2_plot = np.linspace(xlimits[1, 0], xlimits[1, 1], n_grid)

```

```

G1_grid, G2_grid = np.meshgrid(g1_plot, g2_plot)
X_grid = np.column_stack((G1_grid.ravel(), G2_grid.ravel()))

TR_grid = sm_TR.predict_values(X_grid).reshape(G1_grid.shape)
PD_grid = sm_PD.predict_values(X_grid).reshape(G1_grid.shape)

fig = plt.figure(figsize=(16, 12))

# TR 3D surface
ax1 = fig.add_subplot(2, 2, 1, projection='3d')
surf1 = ax1.plot_surface(
    G1_grid, G2_grid, TR_grid,
    cmap='coolwarm',
    edgecolor='none',
    alpha=0.85
)
ax1.scatter(X[:, 0], X[:, 1], y_TR, c='black', s=20, alpha=0.7)
ax1.set_title("TR surrogate surface")
ax1.set_xlabel("G1")
ax1.set_ylabel("G2")
ax1.set_zlabel("TR")
fig.colorbar(surf1, ax=ax1, shrink=0.5, aspect=12)

# TR parity plot
ax2 = fig.add_subplot(2, 2, 2)
ax2.scatter(y_TR, y_TR_pred, s=35, alpha=0.7, color='steelblue')
mn = min(y_TR.min(), y_TR_pred.min())
mx = max(y_TR.max(), y_TR_pred.max())
ax2.plot([mn, mx], [mn, mx], 'r--', linewidth=2)
ax2.set_title(f"TR parity plot (RMSE = {rmse_TR:.4e})")
ax2.set_xlabel("True TR")
ax2.set_ylabel("Predicted TR")
ax2.grid(True, alpha=0.3)

# PD 3D surface
ax3 = fig.add_subplot(2, 2, 3, projection='3d')
surf2 = ax3.plot_surface(
    G1_grid, G2_grid, PD_grid,
    cmap='viridis',
    edgecolor='none',
    alpha=0.85
)
ax3.scatter(X[:, 0], X[:, 1], y_PD, c='black', s=20, alpha=0.7)
ax3.set_title("PD surrogate surface")
ax3.set_xlabel("G1")
ax3.set_ylabel("G2")
ax3.set_zlabel("PD")
fig.colorbar(surf2, ax=ax3, shrink=0.5, aspect=12)

# PD parity plot
ax4 = fig.add_subplot(2, 2, 4)
ax4.scatter(y_PD, y_PD_pred, s=35, alpha=0.7, color='darkgreen')

```

```

mn = min(y_PD.min(), y_PD_pred.min())
mx = max(y_PD.max(), y_PD_pred.max())
ax4.plot([mn, mx], [mn, mx], 'r--', linewidth=2)
ax4.set_title(f"PD parity plot (RMSE = {rmse_PD:.4e})")
ax4.set_xlabel("True PD")
ax4.set_ylabel("Predicted PD")
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
def objective_TR(x):
    x = np.atleast_2d(x)
    return sm_TR.predict_values(x)[0, 0]

result_TR = differential_evolution(
    objective_TR,
    bounds=xlimits,
    seed=42,
    maxiter=300,
    popsize=20,
    tol=1e-6
)

x_opt_TR = result_TR.x
TR_opt = result_TR.fun
PD_at_TR_opt = sm_PD.predict_values(x_opt_TR.reshape(1, -1))[0, 0]

print("="*80)
print("Single-objective optimisation: minimise TR")
print("="*80)
print(f"Optimal G1 = {x_opt_TR[0]:.4f}")
print(f"Optimal G2 = {x_opt_TR[1]:.4f}")
print(f"TR optimum = {TR_opt:.6f}")
print(f"PD at TR optimum = {PD_at_TR_opt:.6f}")
def objective_PD(x):
    x = np.atleast_2d(x)
    return sm_PD.predict_values(x)[0, 0]

result_PD = differential_evolution(
    objective_PD,
    bounds=xlimits,
    seed=42,
    maxiter=300,
    popsize=20,
    tol=1e-6
)

x_opt_PD = result_PD.x
PD_opt = result_PD.fun
TR_at_PD_opt = sm_TR.predict_values(x_opt_PD.reshape(1, -1))[0, 0]

print("="*80)

```

```

print("Single-objective optimisation: minimise PD")
print("="*80)
print(f'Optimal G1 = {x_opt_PD[0]:.4f}')
print(f'Optimal G2 = {x_opt_PD[1]:.4f}')
print(f'PD optimum = {PD_opt:.6f}')
print(f'TR at PD optimum = {TR_at_PD_opt:.6f}')
# CELL 7 — 3D plots of the single-objective optimisation points for TR and PD

from mpl_toolkits.mplot3d import Axes3D

# --- helper for safe prediction on a grid ---
def predict_on_grid(sm, X1, X2):
    Xg = np.column_stack([X1.ravel(), X2.ravel()])
    Z = sm.predict_values(Xg).reshape(X1.shape)
    return Z

# --- design grid ---
n_grid = 80
g1 = np.linspace(xlimits[0, 0], xlimits[0, 1], n_grid)
g2 = np.linspace(xlimits[1, 0], xlimits[1, 1], n_grid)
G1_grid, G2_grid = np.meshgrid(g1, g2)

Z_TR = predict_on_grid(sm_TR, G1_grid, G2_grid)
Z_PD = predict_on_grid(sm_PD, G1_grid, G2_grid)

# -----
# If your optimisation cells already created these variables, keep them.
# Otherwise these fallback blocks will compute them.
# -----

if 'x_opt_TR' not in globals():
    def obj_TR(x):
        return float(sm_TR.predict_values(np.array(x).reshape(1, -1))[0, 0])

    res_TR = differential_evolution(
        obj_TR,
        bounds=[tuple(xlimits[0]), tuple(xlimits[1])],
        seed=42
    )
    x_opt_TR = res_TR.x
    y_opt_TR = res_TR.fun
else:
    y_opt_TR = float(sm_TR.predict_values(np.array(x_opt_TR).reshape(1, -1))[0, 0])

if 'x_opt_PD' not in globals():
    def obj_PD(x):
        return float(sm_PD.predict_values(np.array(x).reshape(1, -1))[0, 0])

    res_PD = differential_evolution(
        obj_PD,
        bounds=[tuple(xlimits[0]), tuple(xlimits[1])],
        seed=42

```

```

    )
    x_opt_PD = res_PD.x
    y_opt_PD = res_PD.fun
else:
    y_opt_PD = float(sm_PD.predict_values(np.array(x_opt_PD).reshape(1, -1))[0, 0])

# --- figure ---
fig = plt.figure(figsize=(16, 7))

# ===== TR surface =====
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
surf1 = ax1.plot_surface(G1_grid, G2_grid, Z_TR, cmap='viridis', alpha=0.85,
edgecolor='none')
ax1.scatter(
    x_opt_TR[0], x_opt_TR[1], y_opt_TR,
    color='red', s=120, marker='*', label='TR optimum'
)
ax1.set_title('Single-objective optimisation for TR')
ax1.set_xlabel('G1')
ax1.set_ylabel('G2')
ax1.set_zlabel('TR')
ax1.legend()
fig.colorbar(surf1, ax=ax1, shrink=0.7, pad=0.08)

# annotate optimum
ax1.text(
    x_opt_TR[0], x_opt_TR[1], y_opt_TR,
    f' Optimum\n G1={x_opt_TR[0]:.3f}\n G2={x_opt_TR[1]:.3f}\n TR={y_opt_TR:.4f}',
    color='black'
)

# ===== PD surface =====
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
surf2 = ax2.plot_surface(G1_grid, G2_grid, Z_PD, cmap='plasma', alpha=0.85,
edgecolor='none')
ax2.scatter(
    x_opt_PD[0], x_opt_PD[1], y_opt_PD,
    color='red', s=120, marker='*', label='PD optimum'
)
ax2.set_title('Single-objective optimisation for PD')
ax2.set_xlabel('G1')
ax2.set_ylabel('G2')
ax2.set_zlabel('PD')
ax2.legend()
fig.colorbar(surf2, ax=ax2, shrink=0.7, pad=0.08)

# annotate optimum
ax2.text(
    x_opt_PD[0], x_opt_PD[1], y_opt_PD,
    f' Optimum\n G1={x_opt_PD[0]:.3f}\n G2={x_opt_PD[1]:.3f}\n PD={y_opt_PD:.2f}',
    color='black'
)

```

```

plt.tight_layout()
plt.show()

print("Single-objective optimum for TR:")
print(f" G1 = {x_opt_TR[0]:.6f}, G2 = {x_opt_TR[1]:.6f}, TR = {y_opt_TR:.6f}")

print("\nSingle-objective optimum for PD:")
print(f" G1 = {x_opt_PD[0]:.6f}, G2 = {x_opt_PD[1]:.6f}, PD = {y_opt_PD:.6f}")
def evaluate_objectives(x):
    x = np.atleast_2d(x)
    tr_val = sm_TR.predict_values(x)[0, 0]
    pd_val = sm_PD.predict_values(x)[0, 0]
    return tr_val, pd_val

weights = np.linspace(0, 1, 50)
solutions = []

for i, w in enumerate(weights):
    def weighted_objective(x):
        x = np.atleast_2d(x)
        tr_val = sm_TR.predict_values(x)[0, 0]
        pd_val = sm_PD.predict_values(x)[0, 0]

        tr_norm = (tr_val - y_TR.min()) / (y_TR.max() - y_TR.min())
        pd_norm = (pd_val - y_PD.min()) / (y_PD.max() - y_PD.min())

        return w * tr_norm + (1 - w) * pd_norm

    result = differential_evolution(
        weighted_objective,
        bounds=xlimits,
        seed=42 + i,
        maxiter=200,
        popsize=15,
        tol=1e-6
    )

    x_sol = result.x
    tr_val, pd_val = evaluate_objectives(x_sol)

    solutions.append({
        "weight": w,
        "G1": x_sol[0],
        "G2": x_sol[1],
        "TR": tr_val,
        "PD": pd_val
    })

pareto_df = pd.DataFrame(solutions)
print(f"Generated {len(pareto_df)} candidate multi-objective solutions")
def is_dominated(row, df):

```

```

for _, other in df.iterrows():
    if ((other["TR"] <= row["TR"]) and (other["PD"] <= row["PD"]) and
        ((other["TR"] < row["TR"]) or (other["PD"] < row["PD"]))) :
        return True
    return False

pareto_mask = [not is_dominated(row, pareto_df) for _, row in pareto_df.iterrows()]
pareto_optimal_df = pareto_df.loc[pareto_mask].copy()
pareto_optimal_df = pareto_optimal_df.sort_values("TR").reset_index(drop=True)

print(f"Non-dominated solutions: {len(pareto_optimal_df)}")
print(pareto_optimal_df.head())
# Cell 10: 3D design-space visualisation and comparison of optimal designs
from mpl_toolkits.mplot3d import Axes3D

# Choose one representative balanced Pareto point
mid_idx = len(pareto_optimal_df) // 2
balanced_point = pareto_optimal_df.iloc[mid_idx]

fig = plt.figure(figsize=(18, 7))

# -----
# Plot 1: 3D design space using TR surface with Pareto solutions
# -----
ax1 = fig.add_subplot(1, 2, 1, projection='3d')

surf1 = ax1.plot_surface(
    G1_grid, G2_grid, TR_grid,
    cmap='coolwarm',
    edgecolor='none',
    alpha=0.65
)

# Original CFD points
ax1.scatter(
    X[:, 0], X[:, 1], y_TR,
    c='black', s=18, alpha=0.5, label='CFD data'
)

# Pareto solutions shown on TR surface
ax1.scatter(
    pareto_optimal_df["G1"],
    pareto_optimal_df["G2"],
    pareto_optimal_df["TR"],
    c='blue', s=45, alpha=0.9, label='Pareto solutions'
)

# Single-objective TR optimum
ax1.scatter(
    x_opt_TR[0], x_opt_TR[1], TR_opt,
    c='red', s=220, marker='*',
    edgecolors='black', linewidths=1.5, label='Min TR optimum'
)

```



```

)

# Single-objective PD optimum shown with its TR value
ax1.scatter(
    x_opt_PD[0], x_opt_PD[1], TR_at_PD_opt,
    c='lime', s=220, marker='*',
    edgecolors='black', linewidths=1.5, label='Min PD optimum'
)

# Balanced Pareto point
ax1.scatter(
    balanced_point["G1"], balanced_point["G2"], balanced_point["TR"],
    c='gold', s=180, marker='D',
    edgecolors='black', linewidths=1.5, label='Balanced Pareto point'
)

ax1.set_title("3D design space with Pareto solutions")
ax1.set_xlabel("G1")
ax1.set_ylabel("G2")
ax1.set_zlabel("TR")
ax1.view_init(elev=25, azim=135)
fig.colorbar(surf1, ax=ax1, shrink=0.6, aspect=12, pad=0.08)

# -----
# Plot 2: Comparison of single-objective optima in objective space
# -----
ax2 = fig.add_subplot(1, 2, 2)

# All non-dominated solutions
ax2.plot(
    pareto_optimal_df["TR"],
    pareto_optimal_df["PD"],
    'o-', color='steelblue', alpha=0.8, linewidth=2,
    label='Pareto front'
)

# Min TR optimum
ax2.scatter(
    TR_opt, PD_at_TR_opt,
    c='red', s=260, marker='*',
    edgecolors='black', linewidths=1.5, label='Min TR optimum', zorder=10
)

# Min PD optimum
ax2.scatter(
    TR_at_PD_opt, PD_opt,
    c='lime', s=260, marker='*',
    edgecolors='black', linewidths=1.5, label='Min PD optimum', zorder=10
)

# Balanced Pareto point
ax2.scatter(

```

```

    balanced_point["TR"], balanced_point["PD"],
    c='gold', s=180, marker='D',
    edgecolors='black', linewidths=1.5, label='Balanced Pareto point', zorder=10
)

# Annotate points
ax2.annotate(
    "Min TR",
    (TR_opt, PD_at_TR_opt),
    xytext=(8, 8),
    textcoords='offset points'
)
ax2.annotate(
    "Min PD",
    (TR_at_PD_opt, PD_opt),
    xytext=(8, 8),
    textcoords='offset points'
)
ax2.annotate(
    "Balanced",
    (balanced_point["TR"], balanced_point["PD"]),
    xytext=(8, 8),
    textcoords='offset points'
)

ax2.set_title("Comparison of optimal designs in objective space")
ax2.set_xlabel("Thermal Resistance (TR)")
ax2.set_ylabel("Pressure Drop (PD)")
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.tight_layout()
plt.show()

# -----
# Print comparison table
# -----
comparison_df = pd.DataFrame({
    "Design": ["Min TR", "Min PD", "Balanced Pareto"],
    "G1": [x_opt_TR[0], x_opt_PD[0], balanced_point["G1"]],
    "G2": [x_opt_TR[1], x_opt_PD[1], balanced_point["G2"]],
    "TR": [TR_opt, TR_at_PD_opt, balanced_point["TR"]],
    "PD": [PD_at_TR_opt, PD_opt, balanced_point["PD"]]
})

print("="*90)
print("COMPARISON OF OPTIMAL DESIGNS")
print("="*90)
print(comparison_df.to_string(index=False))
print("="*90)

```

APPENDIX D: EGO ON DATASET CODE

```
import numpy as np
import matplotlib.pyplot as plt
import warnings, os
warnings.filterwarnings('ignore')

from smt.applications import EGO
from smt.design_space import DesignSpace
from smt.surrogate_models import KRG
from scipy.interpolate import RBFInterpolator
from scipy.stats import norm

os.makedirs('chapter6_figs', exist_ok=True)
print("Imports OK")

data = np.loadtxt("/heatsink.txt")

G1 = data[:, 0:1]
G2 = data[:, 1:2]
TR = data[:, 2:3]
PD = data[:, 3:4]

G1_min, G1_max = G1.min(), G1.max()
G2_min, G2_max = G2.min(), G2.max()

G1_n = (G1 - G1_min) / (G1_max - G1_min)
G2_n = (G2 - G2_min) / (G2_max - G2_min)

X_all = np.hstack([G1_n, G2_n]) # (n, 2)
Y_TR = TR # (n, 1)
Y_PD = PD # (n, 1)

xlimits = np.array([[0., 1.], [0., 1.]])
design_space = DesignSpace(xlimits, seed=42)

n = X_all.shape[0]
print(f'Loaded {n} points from heatsink.txt')
print(f'G1 range : [{G1.min():.4f}, {G1.max():.4f}]')
print(f'G2 range : [{G2.min():.4f}, {G2.max():.4f}]')
print(f'TR range : [{TR.min():.6f}, {TR.max():.6f}]')
print(f'PD range : [{PD.min():.4f}, {PD.max():.4f}]')

# DOE size = half the dataset (floor), rest forms the oracle pool
n_doe = n // 2

np.random.seed(42)
doe_idx = np.random.choice(n, size=n_doe, replace=False)

xdoe = X_all[doe_idx]
ydoe_TR = Y_TR[doe_idx]
ydoe_PD = Y_PD[doe_idx]
```

```

# RBF interpolator spans the full dataset as the high-fidelity oracle
rbf_TR = RBFInterpolator(X_all, Y_TR.ravel(), kernel='thin_plate_spline')
rbf_PD = RBFInterpolator(X_all, Y_PD.ravel(), kernel='thin_plate_spline')

def oracle_TR(x):
    return rbf_TR(np.atleast_2d(x)).reshape(-1, 1)

def oracle_PD(x):
    return rbf_PD(np.atleast_2d(x)).reshape(-1, 1)

print(f'DOE: {n_doe} pts | Oracle pool: {n - n_doe} pts')

N_ITER = n - n_doe # use remaining points as iteration budget
SEED = 42

ego_TR = EGO(
    n_iter = N_ITER,
    criterion = 'EI',
    xdoe = xdoe,
    ydoe = ydoe_TR,
    surrogate = KRG(design_space=design_space, print_global=False),
    n_start = 20,
    seed = SEED,
)

x_opt_TR, y_opt_TR, _, x_data_TR, y_data_TR = ego_TR.optimize(fun=oracle_TR)
print(f'TR optimum : TR = {y_opt_TR.item():.6f} K/W at x_norm = {x_opt_TR.ravel()}')

# Denormalise optimal point back to physical units
G1_opt_TR = x_opt_TR.ravel()[0] * (G1_max - G1_min) + G1_min
G2_opt_TR = x_opt_TR.ravel()[1] * (G2_max - G2_min) + G2_min
print(f'      : G1 = {G1_opt_TR:.4f}, G2 = {G2_opt_TR:.4f}')

ego_PD = EGO(
    n_iter = N_ITER,
    criterion = 'EI',
    xdoe = xdoe,
    ydoe = ydoe_PD,
    surrogate = KRG(design_space=design_space, print_global=False),
    n_start = 20,
    seed = SEED,
)

x_opt_PD, y_opt_PD, _, x_data_PD, y_data_PD = ego_PD.optimize(fun=oracle_PD)
print(f'PD optimum : PD = {y_opt_PD.item():.3f} Pa at x_norm = {x_opt_PD.ravel()}')

# Denormalise optimal point back to physical units
G1_opt_PD = x_opt_PD.ravel()[0] * (G1_max - G1_min) + G1_min
G2_opt_PD = x_opt_PD.ravel()[1] * (G2_max - G2_min) + G2_min
print(f'      : G1 = {G1_opt_PD:.4f}, G2 = {G2_opt_PD:.4f}')

```

```

def best_trace(y_data, n_doe, n_iter):
    """Non-increasing best-found objective at each iteration."""
    return np.array([np.min(y_data[:n_doe + k]) for k in range(n_iter + 1)])

best_TR = best_trace(y_data_TR, n_doe, N_ITER)
best_PD = best_trace(y_data_PD, n_doe, N_ITER)
iters = np.arange(N_ITER + 1)

def loo_rmse_trace(x_data, y_data, n_doe, n_iter):
    """
    LOO-RMSE at each EGO snapshot.
    Computes a full leave-one-out refit at every iteration step.
    """
    ds = DesignSpace(np.array([[0., 1.], [0., 1.]]), seed=0)
    rmse = []
    for k in range(n_doe, n_doe + n_iter + 1):
        xk, yk = x_data[:k], y_data[:k]
        errs = []
        for i in range(len(xk)):
            xi = np.delete(xk, i, axis=0)
            yi = np.delete(yk, i, axis=0)
            m = KRG(design_space=ds, print_global=False)
            m.set_training_values(xi, yi)
            m.train()
            pred = m.predict_values(xk[i:i+1]).ravel()[0]
            errs.append((pred - yk[i, 0])**2)
        rmse.append(np.sqrt(np.mean(errs)))
    return np.array(rmse)

print("Computing LOO-RMSE traces...")
loo_TR = loo_rmse_trace(x_data_TR, y_data_TR, n_doe, N_ITER)
loo_PD = loo_rmse_trace(x_data_PD, y_data_PD, n_doe, N_ITER)
print("Done.")

c = {'TR': '#01696f', 'PD': '#d19900'}
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].plot(iters, best_TR, 'o-', color=c['TR'], lw=2, ms=5)
axes[0].axhline(best_TR[-1], color=c['TR'], lw=1, ls='--', alpha=0.5)
axes[0].set(xlabel='EGO Iteration', ylabel='Best TR (K/W)',
            title='(a) TR Convergence')
axes[0].grid(alpha=0.3)

axes[1].plot(iters, best_PD, 's-', color=c['PD'], lw=2, ms=5)
axes[1].axhline(best_PD[-1], color=c['PD'], lw=1, ls='--', alpha=0.5)
axes[1].set(xlabel='EGO Iteration', ylabel='Best PD (Pa)',
            title='(b) PD Convergence')
axes[1].grid(alpha=0.3)

fig.suptitle('Figure 6.1: Best Objective vs. EGO Iteration', y=1.02)
plt.tight_layout()
plt.show()

```

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].plot(iters, loo_TR, 'o-', color=c['TR'], lw=2, ms=5)
axes[0].set(xlabel='EGO Iteration', ylabel='LOO-RMSE',
            title='(a) Kriging LOO-RMSE — TR')
axes[0].grid(alpha=0.3)

axes[1].plot(iters, loo_PD, 's-', color=c['PD'], lw=2, ms=5)
axes[1].set(xlabel='EGO Iteration', ylabel='LOO-RMSE',
            title='(b) Kriging LOO-RMSE — PD')
axes[1].grid(alpha=0.3)

fig.suptitle('Figure 6.2: Leave-One-Out RMSE vs. EGO Iteration', y=1.02)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

for ax, xd, lbl in [(axes[0], x_data_TR, 'TR'), (axes[1], x_data_PD, 'PD')]:
    ax.scatter(xd[:n_doe, 0], xd[:n_doe, 1],
               c='lightgray', edgecolors='k', s=60, zorder=3, label='Initial DOE')
    sc = ax.scatter(xd[n_doe:, 0], xd[n_doe:, 1],
                   c=np.arange(N_ITER), cmap='viridis',
                   edgecolors='k', s=90, zorder=4, label='EGO infill')
    plt.colorbar(sc, ax=ax, label='Iteration')
    # Denormalise tick labels to physical units
    ax.set_xticks([0, 0.25, 0.5, 0.75, 1.0])
    ax.set_xticklabels([f'{v:.0f}' for v in
                        np.linspace(G1_min, G1_max, 5)])
    ax.set_yticks([0, 0.25, 0.5, 0.75, 1.0])
    ax.set_yticklabels([f'{v:.2f}' for v in
                        np.linspace(G2_min, G2_max, 5)])
    ax.set(xlabel='G1', ylabel='G2', title=f'Infill locations — {lbl}')
    ax.legend(fontsize=8)

fig.suptitle('Figure 6.3: EGO Infill Locations in G1–G2 Space', y=1.02)
plt.tight_layout()
plt.show()

grid_res = 40
g1v = np.linspace(0, 1, grid_res)
g2v = np.linspace(0, 1, grid_res)
G1G, G2G = np.meshgrid(g1v, g2v)
X_grid = np.column_stack([G1G.ravel(), G2G.ravel()])

# Physical axis labels for contour plots
g1_ticks = np.linspace(0, 1, 5)
g1_ticklbls = [f'{v:.0f}' for v in np.linspace(G1_min, G1_max, 5)]
g2_ticks = np.linspace(0, 1, 5)
g2_ticklbls = [f'{v:.2f}' for v in np.linspace(G2_min, G2_max, 5)]

```

```

snaps = [1, 5, 10]
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
ds2 = DesignSpace(xlimits, seed=0)

for col, snap in enumerate(snaps):
    for row, (xd, yd, lbl) in enumerate([
        (x_data_TR, y_data_TR, 'TR'),
        (x_data_PD, y_data_PD, 'PD')
    ]):
        k = n_doe + snap
        xk, yk = xd[:k], yd[:k]
        krg = KRG(design_space=ds2, print_global=False)
        krg.set_training_values(xk, yk)
        krg.train()

        mu = krg.predict_values(X_grid)
        var = np.maximum(krg.predict_variances(X_grid), 0)
        sig = np.sqrt(var)
        f_min = yk.min()
        z = (f_min - mu) / (sig + 1e-12)
        ei = np.maximum((f_min - mu)*norm.cdf(z) + sig*norm.pdf(z), 0)
        ei = ei.reshape(grid_res, grid_res)

        ax = axes[row, col]
        cf = ax.contourf(G1G, G2G, ei, levels=20, cmap='YlOrRd')
        ax.scatter(xk[:n_doe, 0], xk[:n_doe, 1], c='gray', s=30, zorder=3)
        ax.scatter(xk[n_doe:, 0], xk[n_doe:, 1], c='blue', marker='*', s=80, zorder=4)
        if col == 2:
            plt.colorbar(cf, ax=ax, label='EI')
        ax.set_xticks(g1_ticks); ax.set_xticklabels(g1_ticklbls, fontsize=7)
        ax.set_yticks(g2_ticks); ax.set_yticklabels(g2_ticklbls, fontsize=7)
        ax.set_title(f'{lbl} — iter {snap}', fontsize=9)
        ax.set_xlabel('G1', fontsize=8)
        ax.set_ylabel('G2', fontsize=8)

fig.suptitle('Figure 6.4: EI Landscape at Iterations 1, 5, 10')
plt.tight_layout()
plt.show()

np.random.seed(99)
n_boot = 20
lhs_TR = np.zeros((n_boot, N_ITER + 1))
lhs_PD = np.zeros((n_boot, N_ITER + 1))

for b in range(n_boot):
    perm = np.random.permutation(n)
    for ki, k in enumerate(range(n_doe, n_doe + N_ITER + 1)):
        idx = perm[:k]
        lhs_TR[b, ki] = Y_TR[idx].min()
        lhs_PD[b, ki] = Y_PD[idx].min()

total_evals = np.arange(n_doe, n_doe + N_ITER + 1)

```

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

for ax, ego_b, lhs_m, lhs_s, lbl, unit, col in [
    (axes[0], best_TR, lhs_TR.mean(0), lhs_TR.std(0), 'TR', 'K/W', c['TR']),
    (axes[1], best_PD, lhs_PD.mean(0), lhs_PD.std(0), 'PD', 'Pa', c['PD'])]:
    ax.plot(total_evals, ego_b, 'o-', color=col, lw=2, ms=5, label='EGO (SMT)')
    ax.plot(total_evals, lhs_m, 's--', color='gray', lw=2, ms=5, label='Static LHS (mean)')
    ax.fill_between(total_evals, lhs_m - lhs_s, lhs_m + lhs_s,
                    color='gray', alpha=0.2, label='LHS  $\pm 1\sigma$ ')
    ax.set(xlabel='Total evaluations', ylabel=f'Best {lbl} ({unit})',
           title=f'EGO vs. Static LHS — {lbl}')
    ax.legend(fontsize=9); ax.grid(alpha=0.3)

fig.suptitle('Figure 6.5: EGO vs. Static LHS Data Efficiency', y=1.02)
plt.tight_layout()
plt.show()

```

— Table 6.1: Convergence Summary

```

import csv

rows = [
    ['Metric', 'Value'],
    ['Dataset size (n)', n],
    ['Initial DOE pts', n_doe],
    ['EGO iterations', N_ITER],
    ['Total evaluations', n_doe + N_ITER],
    ['Initial best TR (K/W)', f'{best_TR[0]:.6f}'],
    ['Final best TR (K/W)', f'{best_TR[-1]:.6f}'],
    ['TR improvement (%)', f'{100*(best_TR[0]-best_TR[-1])/abs(best_TR[0]):.1f}'],
    ['Initial best PD (Pa)', f'{best_PD[0]:.2f}'],
    ['Final best PD (Pa)', f'{best_PD[-1]:.2f}'],
    ['PD improvement (%)', f'{100*(best_PD[0]-best_PD[-1])/abs(best_PD[0]):.1f}'],
    ['Initial LOO-RMSE TR', f'{loo_TR[0]:.6f}'],
    ['Final LOO-RMSE TR', f'{loo_TR[-1]:.6f}'],
    ['TR RMSE reduction (%)', f'{100*(loo_TR[0]-loo_TR[-1])/loo_TR[0]:.1f}'],
    ['Initial LOO-RMSE PD', f'{loo_PD[0]:.4f}'],
    ['Final LOO-RMSE PD', f'{loo_PD[-1]:.4f}'],
    ['PD RMSE reduction (%)', f'{100*(loo_PD[0]-loo_PD[-1])/loo_PD[0]:.1f}'],
]

print("\nTable 6.1 — Convergence Summary")
for r in rows:
    print(f" {r[0]:<30} {r[1]}")

```